

Unit 2

Image Enhancement in Spatial Domain

What are Filters?

- Image processing filters are used to enhance or modify an image by altering the values of its pixels. Filters are mathematical operations that can be applied to an image using convolution or correlation. They can be used to smooth an image, remove noise, sharpen edges, or extract specific features.



- Common types of image processing filters used in image processing include **Gaussian filters** - which blur an image to reduce noise, and **Sobel filters**- which highlight edges in an image.
- Other filters include **median filters**- which remove salt and pepper noise, and
- Laplacian filters- which enhance the contrast of an image.
- Filtering in image processing can be designed to be adaptive, meaning they change their behavior based on the content of the image.
- **Adaptive filters** are useful in situations where the properties of an image vary across different regions. Overall, filters are an important tool in image processing, allowing for the enhancement and modification of digital images.

Original



Conservative Smoothing



Gaussian Filter

◆ 1. Definition

A **Gaussian filter** is a **low-pass filter** that smooths (blurs) an image by reducing noise and detail.

It uses a **Gaussian function** (bell-shaped curve) to compute the transformation, giving more weight to the center pixels and less to distant ones.

◆ 2. Mathematical Formula

For a 2D Gaussian filter, the function is defined as:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Where:

- x, y → coordinates of the pixel relative to the center of the kernel
- σ → **standard deviation**, controlling the spread (amount of blurring)
- $G(x, y)$ → weight assigned to the pixel at position (x, y)

Gaussian Filter

◆ 3. Key Parameters

Parameter	Meaning
Kernel Size ($N \times N$)	Defines the window (e.g., 3×3 , 5×5 , 7×7). Larger = stronger blur.
σ (Sigma)	Controls how much smoothing occurs. Larger $\sigma \rightarrow$ more blur.
Normalization	The kernel weights sum to 1 so image brightness is preserved.

◆ 4. Example: 5×5 Gaussian Kernel ($\sigma = 1$)

$$G = \frac{1}{273} \begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}$$

- Notice the **highest value at the center** (maximum weight).
- Values decrease symmetrically as we move away from the center.

Gaussian Filter

◆ 5. Working Principle

1. The Gaussian kernel is placed over each pixel of the image.
2. Multiply each pixel value under the kernel by the corresponding Gaussian weight.
3. Sum all these weighted values.
4. Replace the center pixel with this sum.
5. Repeat for every pixel in the image.

This **smooths** the image by averaging neighboring pixels with weights decreasing from the center.

◆ 6. Effects on Image

Effect	Explanation
Noise Reduction	Removes Gaussian (random) noise by averaging nearby pixel values.
Detail Loss	Fine edges and small details are blurred.
Edge Preservation	Slightly better than a simple mean filter due to Gaussian weighting.

Gaussian Filter

◆ 7. Gaussian Filtering vs Other Filters

Filter	Type	Effect	Weighting
Mean Filter	Linear	Smooths image equally	Equal weights
Gaussian Filter	Linear	Smooths while preserving structure	Bell-shaped weights
Median Filter	Non-linear	Removes impulse noise	Chooses median value

◆ 8. In Frequency Domain

The Fourier Transform of a Gaussian is **also a Gaussian**.

This makes it ideal for:

- Removing **high-frequency noise** (sharp changes)
- Keeping **low-frequency components** (smooth variations)

So Gaussian filtering acts as a **low-pass filter** in frequency domain.

Gaussian Filter

◆ 9. Applications

- ✓ Image noise reduction
 - ✓ Preprocessing before edge detection (e.g., Canny detector uses Gaussian smoothing)
 - ✓ Image pyramids and scale-space representation
 - ✓ Background blurring and object detection preprocessing
-

◆ 10. In OpenCV (Python)

python

```
import cv2

# Apply Gaussian Blur
blurred = cv2.GaussianBlur(image, (5, 5), sigmaX=1)
```

- `(5,5)` → kernel size
- `sigmaX` → standard deviation in X direction (Y defaults to same)

1. What is Spatial Domain Image Enhancement?

In spatial domain processing, image enhancement techniques operate directly on pixel values of an image to improve its visual quality or highlight specific features.

If the original image is $f(x, y)$, then the processed image is:

$$g(x, y) = T[f(x, y)]$$

Write something...

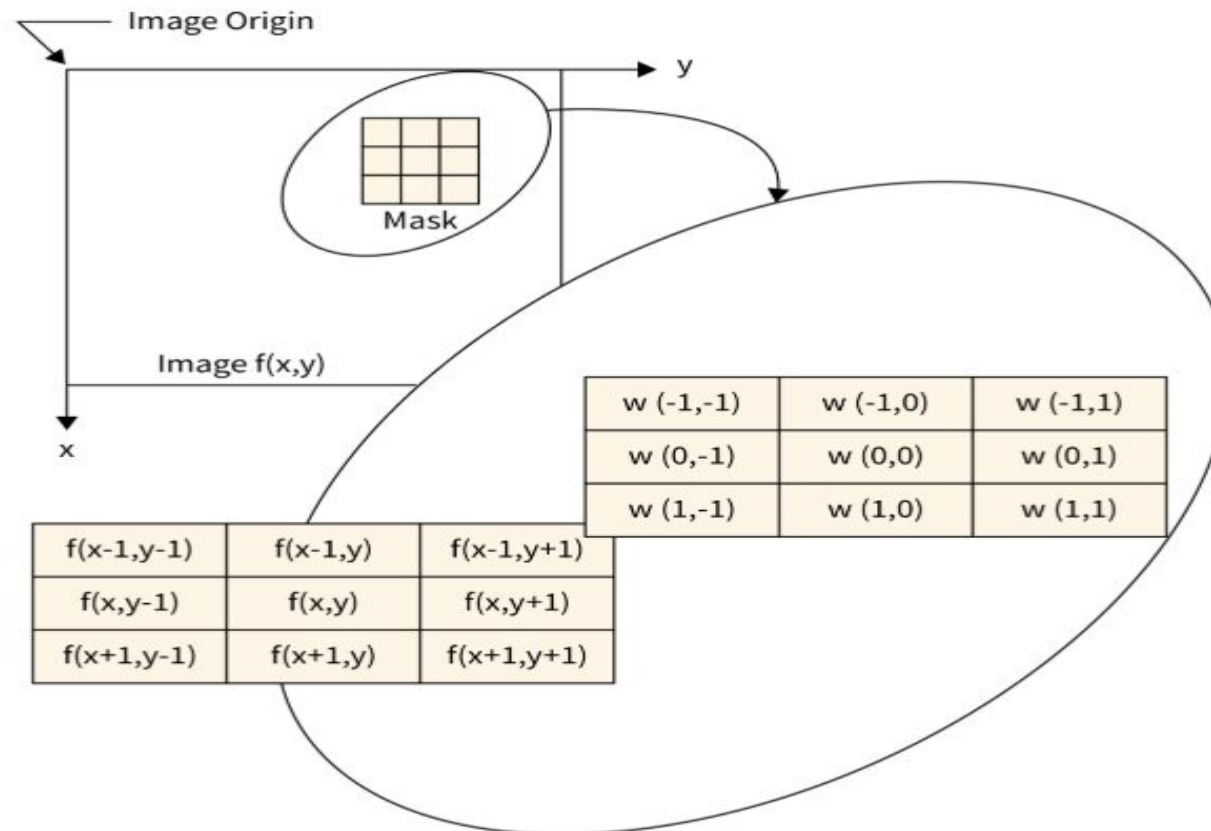
These filters can be applied to an image to achieve various effects, such as smoothing, sharpening, or edge detection

Where:

- $f(x, y) \rightarrow$ original image
- $T \rightarrow$ transformation function
- $g(x, y) \rightarrow$ enhanced image



Linear Spatial Filter-A linear spatial filter operates on an image by applying a weighted sum of neighboring pixel values to each pixel. Common linear filters include Gaussian and Sobel filters, which can be used for smoothing and edge detection, respectively.



Non-linear Spatial Filter

Unlike linear filters, non-linear spatial filtering in image processing operate on an image by applying a non-linear operation to the pixel values. This includes median and max filters, which are used for removing noise and enhancing contrast.

Implementation of Spatial Filtering

[↑ Scroll to top](#)

Here is a sample Python code that uses OpenCV library to implement spatial filtering in image processing:

```
import cv2
import numpy as np
```

```
# Load the image
```

```
img = cv2.imread('input_image.jpg', 0)
```

```
# Define the kernel size and type
```

```
kernel_size = 3
```

```
kernel_type = 'identity'
```

```
# Create the kernel based on the kernel type
```

```
if kernel_type == 'identity':
```

```
    kernel = np.array([[0, 0, 0], [0, 1, 0], [0, 0, 0]])
```

```
elif kernel_type == 'edge_detection':
```

```
    kernel = np.array([[1, 0, -1], [0, 0, 0], [-1, 0, 1]])
```

```
elif kernel_type == 'box_blur':
```

```
    kernel = np.ones((kernel_size, kernel_size), np.float32) / (kernel_size * kernel_size)
```

```
else:
```

```
    print('Invalid kernel type')
```

```
    exit()
```

```
# Apply the kernel to the image
```

```
filtered_img = cv2.filter2D(img, -1, kernel)
```

```
# Display the original and filtered images
```

```
cv2.imshow('Original Image', img)
```

```
cv2.imshow('Filtered Image', filtered_img)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

2. Categories of Spatial Domain Techniques:

- Point Processing (pixel-wise operations)
 - Neighborhood Processing (spatial filtering)
-

1. Point Processing Operations

A. Contrast Stretching

Enhances image contrast by stretching the range of intensity values.

Transformation Equation:

$$s = \frac{(r - r_{\min}) \cdot (s_{\max} - s_{\min})}{r_{\max} - r_{\min}} + s_{\min}$$

Where:

- r : input pixel
- s : output pixel
- $r_{\min}, r_{\max}$: input intensity range
- $s_{\min}, s_{\max}$: output range (e.g., 0 to 255)

Example: If $r_{\min} = 50$, $r_{\max} = 180$, $s_{\min} = 0$, $s_{\max} = 255$:

$$s = \frac{255}{130}(r - 50)$$

B. Logarithmic Transformation

Enhances low-intensity values.

$$s = c \cdot \log(1 + r)$$

Where c is a scaling constant.

C. Power-Law (Gamma) Transformation

Used for brightness control:

$$s = c \cdot r^\gamma$$

- $\gamma < 1$: brightens image
- $\gamma > 1$: darkens image

D. Negative Transformation

Enhances white or gray detail in dark areas.

$$s = L - 1 - r$$

Where L is the number of gray levels (usually 256).

2. Neighborhood (Mask-Based) Processing

A. Smoothing (Low-pass Filtering)

Used to reduce noise.

Averaging Filter (3×3):

$$g(x, y) = \frac{1}{9} \sum_{i=-1}^1 \sum_{j=-1}^1 f(x+i, y+j)$$

Example: For a 3×3 region:

$$\text{Mean} = \frac{100 + 110 + 120 + 130 + 140 + 150 + 160 + 170 + 180}{9} = 140$$

B. Sharpening (High-pass Filtering)

Enhances edges and fine detail.

Laplacian Filter:

$$g(x, y) = f(x, y) - \nabla^2 f(x, y)$$

With kernel:

$$\nabla^2 f = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Example: If pixel center = 100, neighbors = 50 each:

$$g = 100 - [-4 \cdot 100 + 4 \cdot 50] = 100 - (-400 + 200) = 300$$

C. Sobel Edge Detection

Sobel filters detect gradient changes in horizontal and vertical directions.

Horizontal Sobel Kernel G_x :

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Vertical Sobel Kernel G_y :

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Gradient Magnitude:

$$G = \sqrt{G_x^2 + G_y^2}$$

3. Histogram Equalization

Improves image contrast by redistributing the intensity values to span the full range.

Steps:

1. Compute histogram of the image.
2. Normalize it to obtain PDF (Probability Density Function).
3. Compute CDF (Cumulative Distribution Function).
4. Use CDF to remap intensity values.

Transformation Function:

$$s_k = (L - 1) \sum_{j=0}^k p_r(r_j)$$

Where:

- L : number of gray levels (usually 256)
- $p_r(r_j)$: normalized histogram (PDF)

Histogram Equalization

- A digital image is a two-dimensional matrix of two spatial coordinates, with each cell specifying the intensity level of the image at that point. So, we have an $N \times N$ matrix with integer values ranging from a minimum intensity level of 0 to a maximum level of $L-1$, where L denotes the number of intensity levels. Hence, the intensity levels of a pixel r can take on values from $0, 1, 2, 3, \dots (L-1)$. Generally, $L = 2^m$, where m is the number of bits required to represent the intensity levels. Zero level intensity denotes complete black or dark, whereas $L-1$ level indicates complete white or absence of grayscale.

Histogram Equalization

Intensity Transformation:

Intensity transformation is a basic digital image processing technique, where the pixel intensity levels of an image are transformed to new values using a mathematical transformation function, so as to get a new output image. In essence, intensity transformations is simply to implement the following function:

$$s = T(r)$$

where s is the new pixel intensity level and r is the original pixel intensity value of the given image and $r \geq 0$.

With different forms of the transformation function $T(r)$, we get different output images.

Histogram Equalization

Common Intensity Transformation Functions:

1. Image negation: This reverses the grayscales of an image, making dark pixels whiter and white pixels darker. This is completely analogous to the photographic negative, hence the name.

$$s = L - 1 - r$$

2. Log Transform: Here c is some constant. It is used for expanding the dark pixel values in an image.

$$s = c \log(1 + r)$$

3. Power-law Transform: Here c and γ are some arbitrary constants. This transform can be used for a variety of purposes by varying the value of γ .

$$s = cr^\gamma$$

Histogram Equalization

Histogram Equalization:

The histogram of a digital image, with intensity levels between 0 and (L-1), is a function $h(r_k) = n_k$, where r_k is the kth intensity level and n_k is the number of pixels in the image having that intensity level. We can also normalize the histogram by dividing it by the total number of pixels in the image. For an N x N image, we have the following definition of a normalized histogram function:

$$p(r_k) = n_k / N^2$$

This $p(r_k)$ function is the probability of the occurrence of a pixel with the intensity level r_k . Clearly,

$$\sum p(r_k) = 1$$

The histogram of an image, as shown in the figure, consists of the x-axis representing the intensity levels r_k and the y-axis denoting the $h(r_k)$ or the $p(r_k)$ functions.

The histogram of an image gives important information about the grayscale and contrast of the image. If the entire histogram of an image is centered towards the left end of the x-axis, then it implies a dark image. If the histogram is more inclined towards the right end, it signifies a white or bright image. A narrow-width histogram plot at the center of the intensity axis shows a low-contrast image, as it has a few levels of grayscale. On the other hand, an evenly distributed histogram over the entire x-axis gives a high-contrast effect to the image.

Histogram Equalization

In image processing, there frequently arises the need to improve the contrast of the image. In such cases, we use an intensity transformation technique known as histogram equalization. Histogram equalization is the process of uniformly distributing the image histogram over the entire intensity axis by choosing a proper intensity transformation function. Hence, histogram equalization is an intensity transformation process.

Histogram Equalization

Mathematical Derivation of Transformation Function for Histogram Equalization:

Let us consider that the intensity levels of the image r is continuous, unlike the discrete case in digital images. We limit the values that r can take between 0 and $L-1$, that is, $0 \leq r \leq L-1$. $r = 0$ represents black and $r = L-1$ represents white. Let us consider an arbitrary transformation function:

$$s = T(r)$$

where s denotes the intensity levels of the resultant image. We have certain constraints on $T(r)$.

- $T(r)$ must be a strictly increasing function. This makes it an injective function.
- $0 \leq T(r) \leq L-1$. This makes $T(r)$ surjective.

The above two conditions make $T(r)$ a bijective function. We know that such functions are invertible. So we can get back r values from s . We can have a function such that $r = T^{-1}(s)$

Let us now say that the probability density function (pdf) of r is $p_r(x)$ and the cumulative distribution function (CDF) of r is $F_r(x)$. Now the CDF of s will be :

$$F_S(x) = P(s \leq x) = P(T(r) \leq x) = P(r \leq T^{-1}(x)) = F_r(T^{-1}(x)).$$

Histogram Equalization

We put the first condition of $T(r)$ precisely to make the above step hold true. The second condition is needed as s is the intensity value for the output image and so must be between 0 and $(L-1)$.

So, a pdf of s can be obtained by differentiating $F_S(x)$ with respect to x . We get the following relation:

$$p_s(s) = p_r(r) \frac{dr}{ds}$$

Now, if we define the transformation function as follows:

$$s = T(r) = (L - 1) \int_0^r p_r(x) dx$$

Then using this function gives us a uniform pdf for s .

$$\frac{ds}{dr} = (L - 1) \frac{d}{dr} \int_0^r p_r(x) dx = (L - 1) p_r(r)$$

The above step used Leibnitz's integral rule. Using the above derivative, we get:

$$p_s(s) = p_r(r) \frac{dr}{ds} = p_r(r) \frac{1}{(L-1)p_r(r)} = \frac{1}{L-1}$$

So the pdf of s is uniform. This is what we want.

Now, we extend the above continuous case to the discrete case. The natural replacement of the integral sign is the summation. Hence, we are left with the following histogram equalization transformation function.

$$s_k = T(r_k) = (L - 1) \sum_{j=0}^k p_r(r_j) = \frac{(L-1)}{N^2} \sum_{j=0}^k n_j$$

Since s must have integer values, any non-integer value obtained from the above function is rounded off to the nearest integer.

4. Adaptive Filtering

Adapts based on local characteristics of the image, typically using statistical measures in the neighborhood.

Mean Adaptive Filter:

$$g(x, y) = \mu + \frac{\sigma^2 - \sigma_n^2}{\sigma^2} (f(x, y) - \mu)$$

Where:

- μ : local mean
- σ^2 : local variance
- σ_n^2 : noise variance

Summary Table:

Technique	Equation / Kernel	Use Case
Contrast Stretching	$s = \frac{255}{r_{\max} - r_{\min}} (r - r_{\min})$	Low <u>contrast</u> images
Log Transformation	$s = c \cdot \log(1 + r)$	Emphasize dark regions
Gamma Correction	$s = c \cdot r^\gamma$	Brightness tuning
Image Negative	$s = 255 - r$	Dark region enhancement
Smoothing Filter	Mean of 3×3 neighborhood	Noise removal
Laplacian Sharpening	Use <u>∇^2</u> f kernel	Edge enhancement
Sobel Operator	Use G_x, G_y , then magnitude	Edge detection
Histogram Equalization	$s_k = (L - 1) \sum_{j=0}^k p_r(r_j)$	Contrast improvement
Adaptive Filter	See local mean and variance formula	Local noise removal

Feature Extractions – Edge, Corners and
other important features

Edge Detection Operators

- Edge detection is a critical task in image processing and computer vision. It involves identifying and locating sharp discontinuities in an image, which typically correspond to significant changes in intensity or color. These discontinuities are referred to as edges, and they play a crucial role in understanding the structure and contents of an image.

What is image edge detection?

- [Edge detection](#) is a technique used to identify the boundaries of objects within images. It helps in simplifying the image data by reducing the amount of information to be processed while preserving the structural properties of the image. This simplification is essential for various image analysis tasks, including object recognition, segmentation, and image enhancement

Basic Concepts of Edge Detection

Edge detection methods can be broadly categorized into two types: gradient-based methods and second-order derivative methods.

1. Gradient-Based Methods

- Gradient-based methods detect edges by looking for the maximum and minimum in the first derivative of the image. The gradient of an image measures the change in intensity at a point. The most common gradient-based operators are the Sobel, Prewitt, and Roberts Cross operators.

2. Second-Order Derivative Methods

- Second-order derivative methods detect edges by looking for zero crossings in the second derivative of the image. The Laplacian operator is a widely used second-order derivative method. It highlights regions of rapid intensity change, which correspond to edges.

Common Edge Detection Techniques

1. Sobel Operator

- The Sobel operator computes the gradient of the image intensity at each point, giving the direction of the largest possible increase from light to dark and the rate of change in that direction. It uses two 3x3 convolution kernels to calculate the gradient in the horizontal and vertical directions.

Working of the Sobel Operator

- Compute Horizontal and Vertical Gradients:
- Apply the Sobel kernels to the image to obtain the gradients in the x (horizontal) and y (vertical) directions.

$$\text{Sobel Kernel (Horizontal)} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\text{Sobel Kernel (Vertical)} = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

2. Prewitt Operator

Similar to the Sobel operator, the [Prewitt operator](#) calculates the gradient of the image intensity. The main difference is in the convolution kernels used.

Working of the Prewitt Operator

- **Compute Horizontal and Vertical Gradients:** Apply the Prewitt kernels to the image to obtain the gradients in the x and y directions.

$$\text{Prewitt Kernel (Horizontal)} = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\text{Prewitt Kernel (Vertical)} = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

3. Roberts Cross Operator

The Roberts Cross operator performs a simple, quick-to-compute 2x2 gradient measurement on an image. It emphasizes regions of high spatial frequency, which often correspond to edges.

Working of the Roberts Cross Operator

- **Compute Diagonal Gradients:** Apply the Roberts kernels to the image to obtain the gradients along the diagonals.

$$\text{Roberts Kernel (Diagonal 1)} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$\text{Roberts Kernel (Diagonal 2)} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

4. Canny Edge Detector

The Canny edge detector is a multi-stage algorithm that provides a robust solution to edge detection problems. It includes the following steps:

- **Gaussian Blur:** Smooth the image to reduce noise using a Gaussian filter.

$$\text{Gaussian Blur Kernel} = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- **Gradient Calculation:** Compute the intensity gradients of the image using methods like Sobel or Prewitt.
- **Non-Maximum Suppression:** Thin out the edges by suppressing all gradients that are not local maxima.
- **Double Threshold:** Identify strong and weak edges based on threshold values.
- **Edge Tracking by Hysteresis:** Finalize the edge detection by connecting weak edges to strong edges if they are part of the same edge segment.

Implementing Edge Detection in Python

Here's an example of applying the Canny edge detector in Python using OpenCV on Google Colab:

```
# Step 1: Import necessary libraries
```

```
import cv2
import matplotlib.pyplot as plt
from google.colab import files
import numpy as np
```

```
# Step 2: Upload an image
```

```
uploaded = files.upload()
```

```
# Step 3: Read the uploaded image
```

```
image_path = next(iter(uploaded))
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
```

```
# Step 4: Apply Gaussian Blur to reduce noise
```

```
blurred_image = cv2.GaussianBlur(image, (5, 5), 1.4)
```

```
# Step 5: Apply Canny edge detector
```

```
edges = cv2.Canny(blurred_image, 50, 150)
```

```
# Step 6: Display the original image and the edge-detected image
```

```
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.title('Original Image')
plt.imshow(image, cmap='gray')
plt.axis('off')
```

```
plt.subplot(1, 2, 2)
plt.title('Edge Detected Image')
plt.imshow(edges, cmap='gray')
plt.axis('off')
```

```
plt.show()
```

Original Image



Edge Detected Image



Applications of Edge Detection

Edge detection has numerous applications in various fields:

- **Medical Imaging:** Enhancing features in medical images for better diagnosis.
- **Computer Vision:** Object detection, facial recognition, and scene understanding.
- **Robotics:** Enabling robots to perceive and understand their environment.
- **Automotive:** Lane detection in autonomous driving systems.
- **Photography:** Image enhancement and artistic effects.

Challenges in Edge Detection

Edge detection can be challenging due to:

- **Noise:** High levels of noise can cause false edges.
- **Texture:** Complex textures can lead to fragmented or missing edges.
- **Illumination:** Variations in lighting can affect edge detection accuracy.
- **Scale:** Objects of different scales may require different edge detection parameters.

Harris Corner Detection

What is Harris Corner Detection?

- Harris Corner Detection is a **feature detection algorithm** used in computer vision to identify **corner points** (keypoints) in an image. Corners are important because they are stable under translation, rotation, and sometimes even scale changes — making them good features for tasks like **tracking, matching, and object recognition**.

Haris Corner Detection

Steps of Corner Detection

1. Compute image gradients over small region
2. Subtract mean from each image gradient
3. Compute the covariance matrix
4. Compute eigenvectors and eigenvalues
5. Use threshold on eigenvalues to detect corners

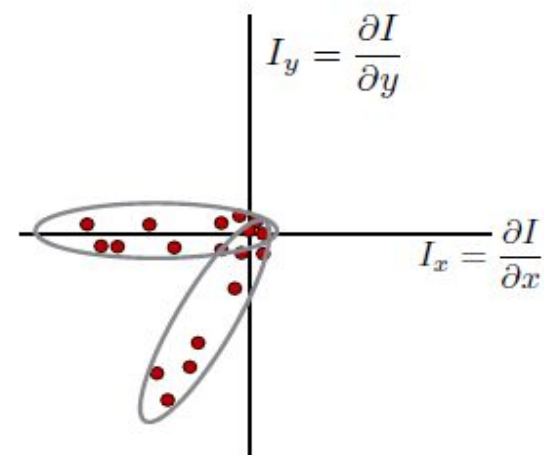
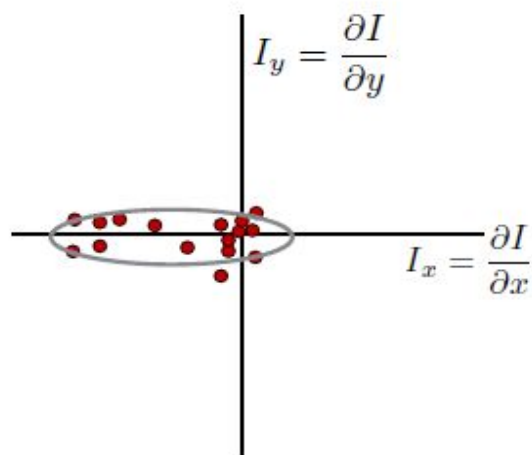
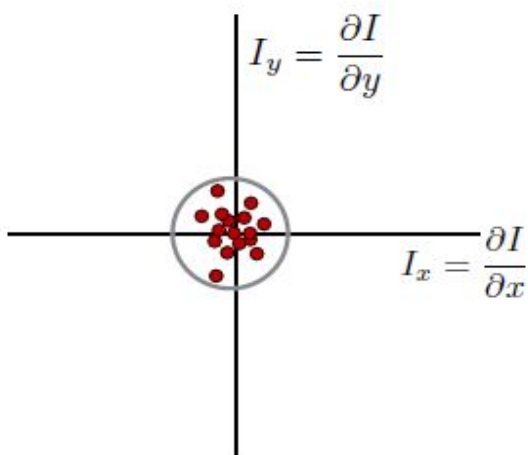
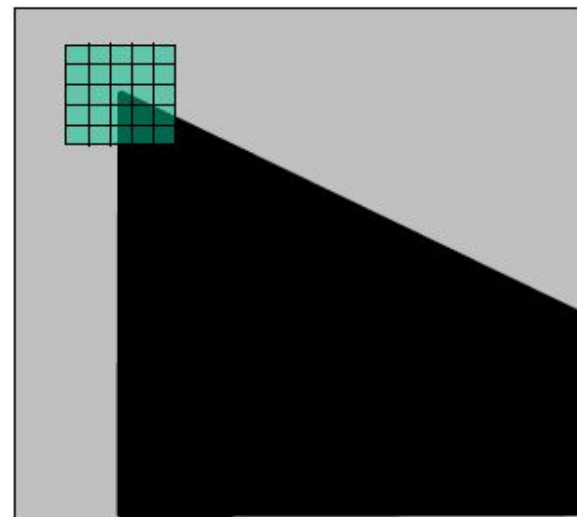
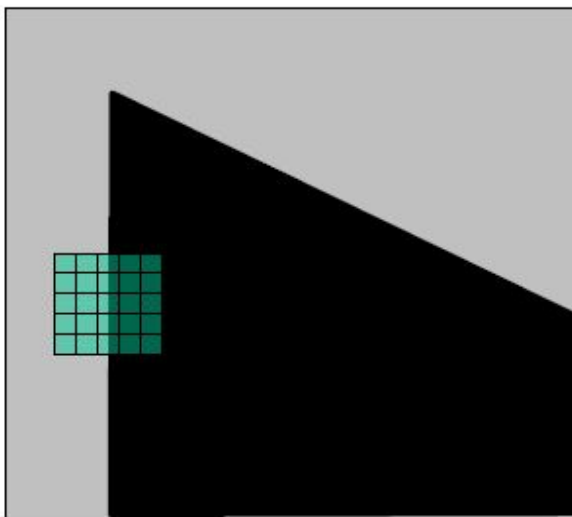
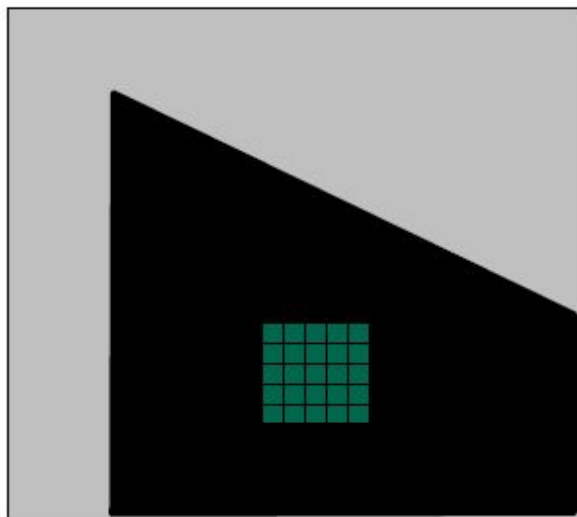
$$I_x = \frac{\partial I}{\partial x}$$



$$I_y = \frac{\partial I}{\partial y}$$



$$\begin{bmatrix} \sum_{p \in P} I_x I_x & \sum_{p \in P} I_x I_y \\ \sum_{p \in P} I_y I_x & \sum_{p \in P} I_y I_y \end{bmatrix}$$



distribution reveals edge orientation and magnitude

Harris Corner Detection

◆ Steps of Harris Corner Detection

1. Compute Image Gradients

- Compute the partial derivatives of the image intensity using Sobel filters:
 - $I_x = \frac{\partial I}{\partial x}$
 - $I_y = \frac{\partial I}{\partial y}$
 - These gradients show how intensity changes in the horizontal and vertical directions.
-

2. Form the Structure Tensor (Auto-correlation Matrix)

For each pixel, construct a 2×2 matrix M :

$$M = \begin{bmatrix} \sum w(x, y) I_x^2 & \sum w(x, y) I_x I_y \\ \sum w(x, y) I_x I_y & \sum w(x, y) I_y^2 \end{bmatrix}$$

- $w(x, y)$ is a window function (often Gaussian).
 - This matrix captures intensity changes in a local neighborhood.
-

Haris Corner Detection

3. Corner Response Function

Instead of directly computing eigenvalues of M , Harris & Stephens proposed a corner response function:

$$R = \det(M) - k \cdot (\text{trace}(M))^2$$

Where:

- $\det(M) = \lambda_1 \lambda_2$
- $\text{trace}(M) = \lambda_1 + \lambda_2$
- k is an empirical constant (usually $0.04 \leq k \leq 0.06$)

Here:

- If R is large and positive $\rightarrow$ Corner
- If R is large and negative $\rightarrow$ Edge
- If R is small (close to 0) $\rightarrow$ Flat region

4. Thresholding

- Apply a threshold to R .
- Keep only points where R is greater than the threshold.

Haris Corner Detection

5. Non-Maximum Suppression

- Corners usually appear in clusters.
 - Keep only the local maxima of R to avoid multiple responses for the same corner.
-

♦ Intuition with Eigenvalues of M

- λ_1, λ_2 are eigenvalues of M .
- **Flat Region:** both small.
- **Edge:** one large, one small.
- **Corner:** both large.

Harris Corner Detection

◆ Advantages of Harris Corner Detector

- Rotation invariant.
- Robust to noise (with Gaussian smoothing).

◆ Limitations

- Not scale-invariant (fails if object size changes).
- Later improved by **Shi-Tomasi** (uses min eigenvalue criterion).
- Further extended in **SIFT, SURF, ORB** etc.

Feature Extraction Method

- Feature extraction transforms raw image data into meaningful representations (features) for tasks such as recognition, matching, and motion analysis.
- **◆ Types of Features**
- **Edges**
 - Represent intensity discontinuities.
 - Methods: Sobel, Canny, Laplacian of Gaussian.
 - Applications: object boundaries, shape recognition.
- **Corners / Interest Points**
 - Points with strong intensity variation in multiple directions.
 - Example: Harris Corner Detector (uses structure tensor).
 - Applications: tracking, motion estimation.

Feature Extraction Method

Blobs / Regions

- Areas with distinct intensity or texture.
- Example: Difference of Gaussian (DoG), MSER(Maximally Stable Extremal Region)
- Applications: object detection.

Descriptors

- Compact numerical representation of local patches.
- Example: SIFT, SURF, ORB.
- Used for feature matching across images.

Transform-based Features

- Fourier Transform, Wavelets, Hough Transform.
- Capture frequency or shape properties.

Desirable Properties of Features

- Repeatability**: detected consistently under different conditions.
- Distinctiveness**: unique enough to match correctly.
- Invariance**: robust to scale, rotation, illumination changes.
- Compactness**: efficient storage and computation.

Feature Extraction Method

◆ SIFT (Scale-Invariant Feature Transform)

SIFT is a **feature detection and description algorithm** developed by David Lowe (1999, published 2004). It is widely used in computer vision tasks such as **object recognition, image stitching, 3D reconstruction, and motion tracking** because it is robust to **scale, rotation, and partial illumination changes**.

Key Properties

- **Scale-invariant** → Features remain stable even if the image is zoomed in/out.
- **Rotation-invariant** → Features do not change with image rotation.
- **Partially affine invariant** → Robust to small viewpoint changes.
- **Distinctive** → Produces unique descriptors that allow reliable matching.

2. SIFT Pipeline

(a) Scale-Space Construction

- To detect keypoints at different scales, SIFT constructs a **scale space** using Gaussian blur.
- The *Difference of Gaussians (DoG)* is computed:

$$D(x, y, \sigma) = G(x, y, k\sigma) * I(x, y) - G(x, y, \sigma) * I(x, y)$$

where G = Gaussian function, I = image, σ = scale.

SIFT

(b) Keypoint Detection

- Local extrema are found in the DoG images (across scale and space).
 - Each pixel is compared with its 26 neighbors (8 in current scale + 9 in scale above + 9 in scale below).
 - Points that are **local maxima or minima** are selected as candidate keypoints.
-

(c) Keypoint Localization

- Candidates are refined by fitting a quadratic function.
 - Low-contrast points and unstable edge responses are removed using the **Hessian matrix test** (like Harris corner rejection).
-

(d) Orientation Assignment

- Each keypoint is assigned one or more orientations using local image gradient directions:

$$m(x, y) = \sqrt{(I_{x+1,y} - I_{x-1,y})^2 + (I_{x,y+1} - I_{x,y-1})^2}$$

$$\theta(x, y) = \tan^{-1} \left(\frac{I_{y+1} - I_{y-1}}{I_{x+1} - I_{x-1}} \right)$$

- Orientation histograms are built around the keypoint.

(e) Descriptor Generation

- Around each keypoint, a **16×16 neighborhood** is taken.
 - It is divided into **4×4 subregions**, each with **8 orientation bins**.
 - This results in a **128-dimensional descriptor vector**.
 - The vector is normalized to reduce illumination effects.
-

3. Applications

- Object recognition (matching features across different views).
- Image stitching (panorama creation).
- 3D scene reconstruction (used in SfM).
- Tracking and motion estimation.

4. Advantages

- Robust to scale, rotation, illumination changes.
 - Produces distinctive, repeatable features.
 - Works well in cluttered or partially occluded images.
-

5. Limitations

- **Computationally expensive** compared to newer methods (SURF, ORB).
- Patented (till 2020), which limited its usage in open-source projects.
- Not ideal for real-time applications (deep learning or ORB are better).

SURF – Speed Up Robust Features

SURF is a **local feature detector and descriptor** proposed by **Herbert Bay et al. (2006)**. It is designed as a **fast and robust alternative to SIFT**, providing:

- **Scale invariance**
- **Rotation invariance**
- **Robustness to illumination changes**
- **High computational speed**

SURF is widely used in:

- Object recognition
- Image stitching
- Tracking
- Registration

Key Idea of SURF

SURF accelerates feature detection and description by:

1. Using **integral images**
2. Approximating **second-order Gaussian derivatives** with **box filters**
3. Using **Hessian matrix** for interest point detection
4. Using **Haar wavelet responses** for descriptors

SURF – Speed Up Robust Features

Integral Image (Speed Optimization)

Definition

Integral Image (Summed-Area Table)

An **integral image** is a transformed version of an image in which **each pixel stores the cumulative sum of all pixel values above and to the left of it** (including itself).

It is mainly used to **compute rectangular region sums extremely fast**, in **constant time $O(1)$** , regardless of the rectangle size.

1. Formal Definition

For a grayscale image $I(x, y)$, the integral image $I_{\Sigma}(x, y)$ is defined as:

$$I_{\Sigma}(x, y) = \sum_{i=0}^x \sum_{j=0}^y I(i, j)$$

This means:

- At position (x, y) , the value is the sum of **all pixels in the rectangle from (0,0) to (x,y)**.

Original Image (4×4)

$$I = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

Integral Image

$$I_{\Sigma} = \begin{bmatrix} 1 & 3 & 6 & 10 \\ 6 & 14 & 24 & 36 \\ 15 & 33 & 54 & 78 \\ 28 & 60 & 96 & 136 \end{bmatrix}$$

2. How It Is Computed (Recursive Form)

The integral image can be computed efficiently using:

$$I_{\Sigma}(x, y) = I(x, y) + I_{\Sigma}(x - 1, y) + I_{\Sigma}(x, y - 1) - I_{\Sigma}(x - 1, y - 1)$$

✦ The subtraction avoids **double-counting** the overlapping region.

SURF – Speed Up Robust Features

2. Hessian Matrix-Based Interest Point Detection

What is done?

SURF detects keypoints by evaluating **second-order intensity changes**.

Why Hessian Matrix?

- Captures **blob-like structures**
- Strong response at corners and edges intersection

Mathematical Form

$$H(x, y, \sigma) = \begin{bmatrix} L_{xx} & L_{xy} \\ L_{xy} & L_{yy} \end{bmatrix}$$

Where:

- $L_{xx} = \frac{\partial^2}{\partial x^2} (G_\sigma * I)$
- $L_{yy} = \frac{\partial^2}{\partial y^2} (G_\sigma * I)$
- $L_{xy} = \frac{\partial^2}{\partial x \partial y} (G_\sigma * I)$

Determinant of Hessian

$$\det(H) = L_{xx}L_{yy} - L_{xy}^2$$

★ Meaning:

- High determinant → strong variation in two directions → good keypoint

3. Box Filter Approximation

What is done?

Gaussian second derivatives are approximated using **box filters**.

Why approximation?

Gaussian convolution is expensive.

Box filters + integral images = **very fast computation**.

Approximate Determinant

$$\det(H_{approx}) = D_{xx}D_{yy} - (wD_{xy})^2$$

Where:

- D_{xx}, D_{yy}, D_{xy} : box filter responses
- $w \approx 0.9$: balances scale difference

★ **Impact:** Enables real-time performance.

SURF – Speed Up Robust Features

4. Scale Space Construction

What is done?

Keypoints are detected at **multiple scales**.

Why scale invariance?

Objects appear larger or smaller depending on distance.

How SURF handles scale

Instead of resizing the image:

- Image size remains constant
- Filter size is increased

$$\sigma \propto \text{filter size}$$

Non-Maximum Suppression

For each point:

- Compare with **26 neighbors**
(8 in image plane + 9 above scale + 9 below scale)

✦ Only local maxima survive

5. Keypoint Localization

What is done?

Refine position and scale of detected keypoints.

Why needed?

Box filters give approximate locations.

Method

Quadratic interpolation in:

- x
- y
- scale

✦ **Effect:** Sub-pixel accuracy.

SURF – Speed Up Robust Features

6. Orientation Assignment

What is done?

Each keypoint is assigned a **dominant orientation**.

Why?

To achieve **rotation invariance**.

Haar Wavelet Responses

Computed in a circular region of radius 6σ :

$$dx = \sum \text{Haar}_x \quad dy = \sum \text{Haar}_y$$

Weighted by Gaussian kernel:

$$w(x, y) = e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

Dominant Direction

A sliding window of 60° finds maximum response vector:

$$\theta = \tan^{-1} \left(\frac{\sum dy}{\sum dx} \right)$$

↓

✦ Descriptor is rotated to this orientation

7. Descriptor Window Construction

What is done?

An oriented square region of size:

$$20\sigma \times 20\sigma$$

Centered on keypoint.

Why this size?

- Large enough to capture context
- Small enough to avoid noise

8. Sub-Region Division

What is done?

Descriptor window is divided into:

$$4 \times 4 = 16 \text{ sub-regions}$$

✦ Each sub-region captures **local gradient structure**.

SURF – Speed Up Robust Features

9. Feature Vector Computation

What is computed per sub-region?

$$v = \left[\sum dx, \sum dy, \sum |dx|, \sum |dy| \right]$$

Why these terms?

Term	Meaning
$\sum dx$	Dominant horizontal direction
$\sum dy$	Dominant vertical direction
$(\sum$	dx
$(\sum$	dy

Final Descriptor Size

$$16 \times 4 = 64 \text{ features}$$

10. Descriptor Normalization

What is done?

Feature vector normalized:

$$\mathbf{v} = \frac{\mathbf{v}}{\|\mathbf{v}\|}$$

Why?

- Illumination invariance
- Contrast robustness

SURF – Speed Up Robust Features

11. Feature Matching

What is done?

Match descriptors between images.

Distance Metric

$$d(\mathbf{f}_1, \mathbf{f}_2) = \sqrt{\sum (f_{1i} - f_{2i})^2}$$

Ratio Test

$$\frac{d_1}{d_2} < 0.8$$

✦ Rejects ambiguous matches.

12. End-to-End SURF Pipeline

1. Compute integral image
2. Apply box filters
3. Compute Hessian determinant
4. Detect scale-space extrema
5. Assign orientation
6. Build descriptor
7. Normalize
8. Match features

SURF Vs SIFT

Difference Between SIFT and SURF

Feature	SIFT	SURF
Keypoint Detection	Difference of Gaussian (DoG)	Determinant of Hessian matrix
Descriptor Size	128-dimensional	64-dimensional
Orientation Assignment	Gradient orientation	Haar wavelet responses
Speed	Slower due to more complex computations	Faster due to integral images and simpler computations
Accuracy	Higher accuracy under extreme transformations	Slightly lower accuracy but faster
Robustness	Highly robust to scale, rotation, and illumination changes	Robust, but slightly less than SIFT under some conditions
Computational Complexity	Higher	Lower
Suitable Applications	Applications requiring high precision	Real-time applications, large-scale processing

ORB (Oriented FAST and Rotated BRIEF)

1. Introduction

ORB is a **fast, free, and efficient feature extraction method** introduced by **Ethan Rublee et al. (2011)**.

It was designed as a **replacement for SIFT and SURF**, avoiding patent restrictions while maintaining good performance.

ORB combines:

- **FAST** → Keypoint detection
- **BRIEF** → Feature description
- **Orientation & rotation handling** → Added by ORB

2. ORB Pipeline (High-Level)

1. Image pyramid construction
2. FAST keypoint detection
3. Keypoint scoring using Harris measure
4. Orientation assignment (Intensity centroid)
5. Rotated BRIEF (rBRIEF) descriptor
6. Feature matching using Hamming distance

3. Image Pyramid (Scale Invariance)

Why needed?

Objects appear at different sizes.

How ORB handles scale

ORB builds an **image pyramid**:

$$I_0, I_1, I_2, \dots, I_n$$

Where:

$$I_{k+1} = \text{Downsample}(I_k)$$

Each level is scaled by factor $s \approx 1.2$.

✚ FAST is applied **independently at each pyramid level**.

4. FAST Keypoint Detection

What is FAST?

FAST (Features from Accelerated Segment Test) detects **corners**.

How FAST Works

For a candidate pixel p :

- Consider **16 pixels on a circle** of radius 3
- Pixel p is a corner if:
 - At least **N contiguous pixels** are either:
 - Brighter than $I(p) + t$, or
 - Darker than $I(p) - t$

$$|I(x_i) - I(p)| > t$$

Typically:

- $N = 12$
- t = threshold

✦ FAST is **extremely fast** but does **not provide orientation or scale**.

5. Keypoint Scoring (Harris Corner Measure)

Why scoring is needed

FAST detects **many unstable points**.

Harris Corner Score

ORB ranks FAST points using Harris response:

$$R = \det(M) - k(\text{trace}(M))^2$$

Where:

$$M = \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

Only **top N keypoints** are retained per pyramid level.

✦ This improves repeatability.

6. Orientation Assignment (Rotation Invariance)

ORB assigns orientation using the **Intensity Centroid Method**.

Image Moments

For a patch around keypoint:

$$m_{pq} = \sum_x \sum_y x^p y^q I(x, y)$$

Centroid Calculation

$$C = \left(\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}} \right)$$

Orientation Angle

$$\theta = \tan^{-1} \left(\frac{m_{01}}{m_{10}} \right)$$

✦ Direction from keypoint to centroid defines orientation.

7. BRIEF Descriptor

What is BRIEF?

BRIEF is a **binary descriptor** based on intensity comparisons.

Binary Test

For a smoothed image patch:

$$\tau(p; x_i, y_i) = \begin{cases} 1 & \text{if } I(x_i) < I(y_i) \\ 0 & \text{otherwise} \end{cases}$$

Each test produces **1 bit**.

Descriptor Construction

$$\text{BRIEF}(p) = \sum_{i=1}^n 2^{i-1} \tau(p; x_i, y_i)$$

Typical sizes:

- 128 bits
- **256 bits (ORB default)**

✦ Very fast and memory efficient.



8. Rotated BRIEF (rBRIEF)

Problem

BRIEF is **not** rotation invariant.

ORB Solution

Rotate sampling pattern using orientation θ :

$$\begin{bmatrix} x'_i \\ y'_i \end{bmatrix} = R_\theta \begin{bmatrix} x_i \\ y_i \end{bmatrix}$$

Where:

$$R_\theta = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

✦ Produces **rotation-invariant** binary descriptors.

9. Descriptor Learning (Improved BRIEF)

ORB improves BRIEF by:

- Removing highly correlated tests
- Maximizing variance of each bit
- Greedy selection of best test pairs

✦ Results in **more discriminative descriptors**.

10. Feature Matching

Distance Metric

ORB uses **Hamming distance**:

$$d_H(a, b) = \sum (a_i \oplus b_i)$$

Why Hamming?

- Binary descriptors
- Extremely fast using bitwise XOR

11. Worked Example

Corner Detection

At pixel p , 12 out of 16 circle pixels satisfy FAST condition

→ p is a keypoint

Orientation

Given:

$$m_{10} = 40, \quad m_{01} = 30$$

$$\theta = \tan^{-1} \left(\frac{30}{40} \right) = 36.87^\circ$$

Binary Test

$$I(x_1) = 120, \quad I(y_1) = 135 \Rightarrow 1$$

Repeat for 256 pairs → 256-bit descriptor.

12. Advantages and Limitations

Advantages

- ✓ Free (no patents)
- ✓ Very fast
- ✓ Suitable for real-time systems
- ✓ Low memory usage

Limitations

- ✗ Less robust to illumination changes
- ✗ Slightly less accurate than SIFT/SURF
- ✗ FAST sensitive to noise

13. ORB vs SURF vs SIFT

Feature	ORB	SURF	SIFT
Detector	FAST	Hessian	DoG
Descriptor	BRIEF (binary)	Haar-based	Gradient histogram
Size	256 bits	64 floats	128 floats
Speed	Fastest	Fast	Slow
Patent	Free	Patented	Expired

14. Applications

- Real-time SLAM
- Mobile vision
- UAV navigation
- Object tracking
- Feature matching under constraints

Principal Component Analysis (PCA) / PCI

1. Introduction

Principal Component Analysis (PCA), sometimes referred to in image processing contexts as Principal Component Identification (PCI), is a dimensionality reduction technique widely used in machine learning, computer vision, and image processing. It transforms data into a new coordinate system where the greatest variance lies on the first coordinate (the first principal component), the second greatest variance on the second coordinate, and so on.

2. Why PCA?

- High-dimensional data often contains redundancy and noise.
- PCA reduces dimensions while preserving maximum variance.
- It improves computational efficiency and removes correlations among features.
- Common in image compression, feature extraction, and noise reduction.

Principal Component Analysis (PCA) / PCI

3. PCA Workflow / Steps

Step 1: Standardize the Data

Ensure data has **zero mean** and **unit variance**:

$$x' = \frac{x - \mu}{\sigma}$$

Step 2: Compute the Covariance Matrix

$$C = \frac{1}{n-1} X^T X$$

where X is the matrix of input data.

Step 3: Eigenvalue Decomposition

Solve for eigenvalues (λ) and eigenvectors (v):

$$Cv = \lambda v$$

- **Eigenvectors** → Principal components (directions of maximum variance).
- **Eigenvalues** → Magnitude of variance in each direction.

Step 3: Eigenvalue Decomposition

Solve for eigenvalues (λ) and eigenvectors (v):

$$Cv = \lambda v$$

- **Eigenvectors** → Principal components (directions of maximum variance).
- **Eigenvalues** → Magnitude of variance in each direction.

Step 4: Select Principal Components

Choose the top k eigenvectors with largest eigenvalues.

This reduces dimensionality from $d \rightarrow k$.

Step 5: Transform Data

Project original data onto new axes:

$$Y = XW$$

where W contains selected eigenvectors.

Principal Component Analysis (PCA) / PCI

4. Numerical Example

Suppose we have 2D data points:

$$X = \{(2, 3), (3, 4), (4, 5), (5, 6)\}$$

- Step 1: Mean = (3.5, 4.5). Subtract mean → Centered data.
- Step 2: Covariance matrix:

$$C = \begin{bmatrix} 1.67 & 1.67 \\ 1.67 & 1.67 \end{bmatrix}$$

- Step 3: Eigenvalues: $\lambda_1 = 3.34, \lambda_2 = 0$.
- Step 4: Principal component = [1, 1].
- Step 5: Data reduces to 1D along [1, 1] axis.

5. Applications in Image Processing

- **Face Recognition (Eigenfaces)** → PCA reduces high-dimensional pixel data into few principal components.
 - **Image Compression** → Stores only top eigenvectors, reducing storage.
 - **Noise Reduction** → Discards low-variance components that represent noise.
 - **Medical Imaging** → Enhances critical features while reducing redundant info.
 - **Motion Estimation / Object Tracking** → Extracts dominant features for analysis.
-

6. Advantages

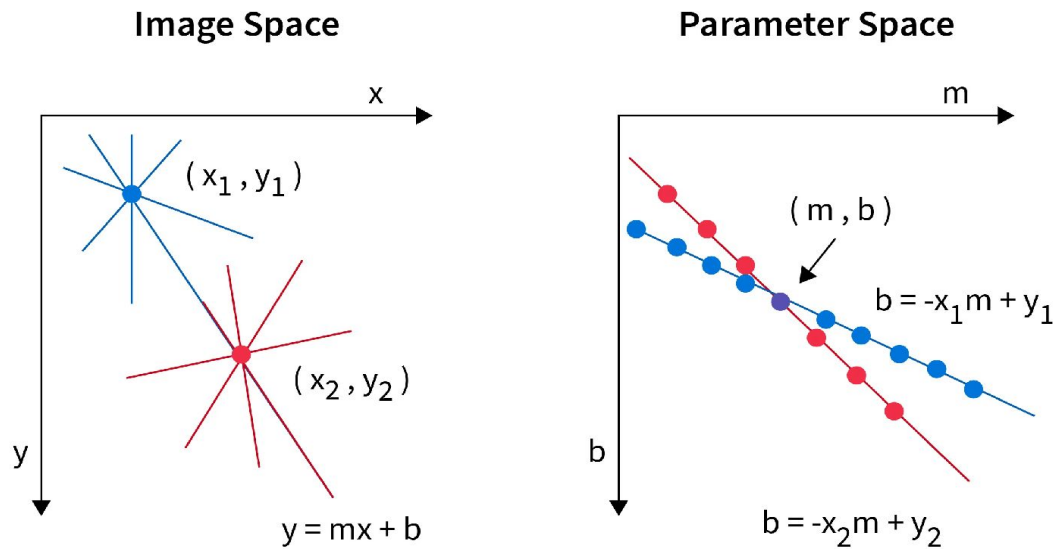
- Reduces dimensionality efficiently.
 - Removes redundancy.
 - Speeds up training and computation.
 - Works well with correlated features.
-

7. Limitations

- Assumes linear relationships.
- Sensitive to scaling (requires normalization).
- Loses interpretability (principal components may not correspond to real-world features).
- Not effective if variance does not capture important ↓ class information (e.g., in classification tasks).

What is Hough Transform?

The Hough transform in image processing is a technique used to detect simple geometric shapes in images. It works by transforming the **image space into a parameter space**, where the geometric shapes can be detected through the identification of patterns in the parameter space.



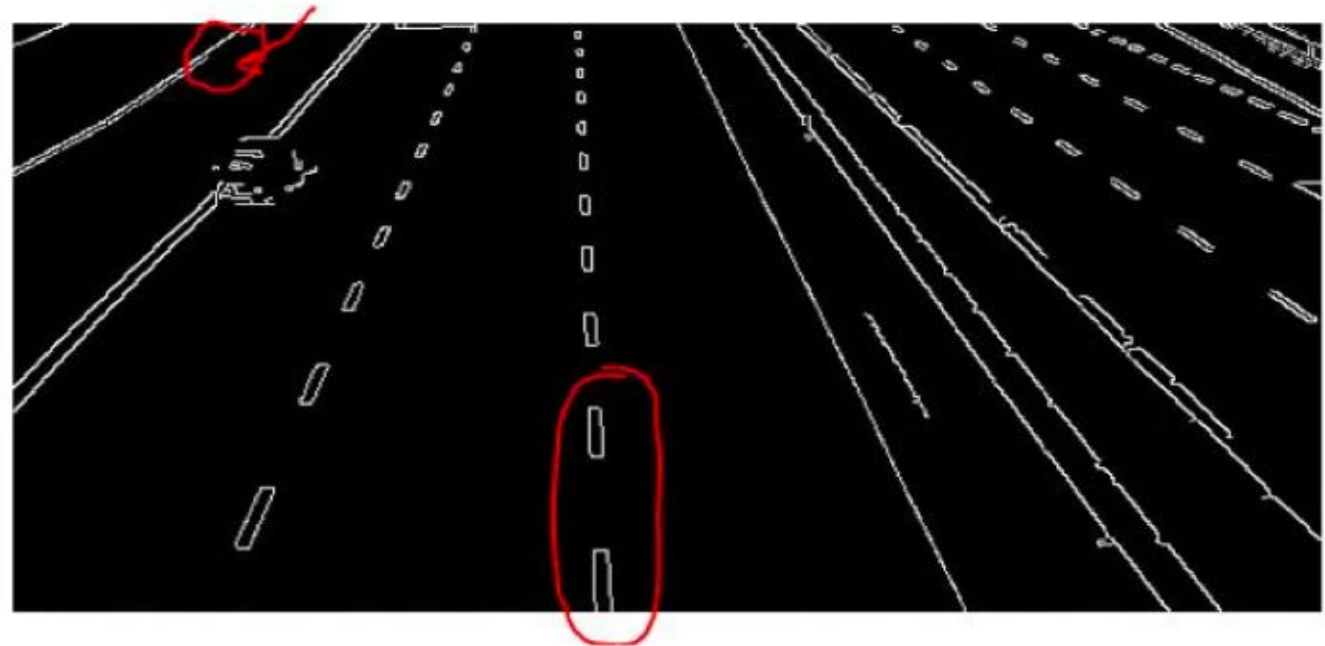
Why is Hough Transform Needed?

The Hough transform is needed because traditional image processing techniques like edge detection and thresholding are not always effective at detecting simple geometric shapes in images. These techniques can be particularly ineffective when the shapes are distorted, incomplete, or partially obscured. The Hough transform can detect these shapes by transforming the image space into a parameter space where the shapes can be more easily identified.

Original Image of the lane



Image after applying edge detection technique. Red circles show that the line is breaking there.



After using Hough Transform



How does it Work?

The Hough transform in image processing works by transforming the image space into a parameter space. For example, in the case of detecting lines in images, the image space is transformed into a parameter space consisting of two parameters: the slope and the y-intercept of the line.

Each pixel in the image space is then mapped to a curve in the parameter space that represents all the possible lines that could pass through that pixel. The curves in the parameter space are then analyzed to detect the presence of lines in the image.

Algorithm

The algorithm of the Hough transform in image processing can be summarized as follows:

- For each pixel in the image, compute all the possible curves in the parameter space that pass through that pixel.
- For each curve in the parameter space, increment a corresponding accumulator array.
- Analyze the accumulator array to detect the presence of simple geometric shapes in the image.

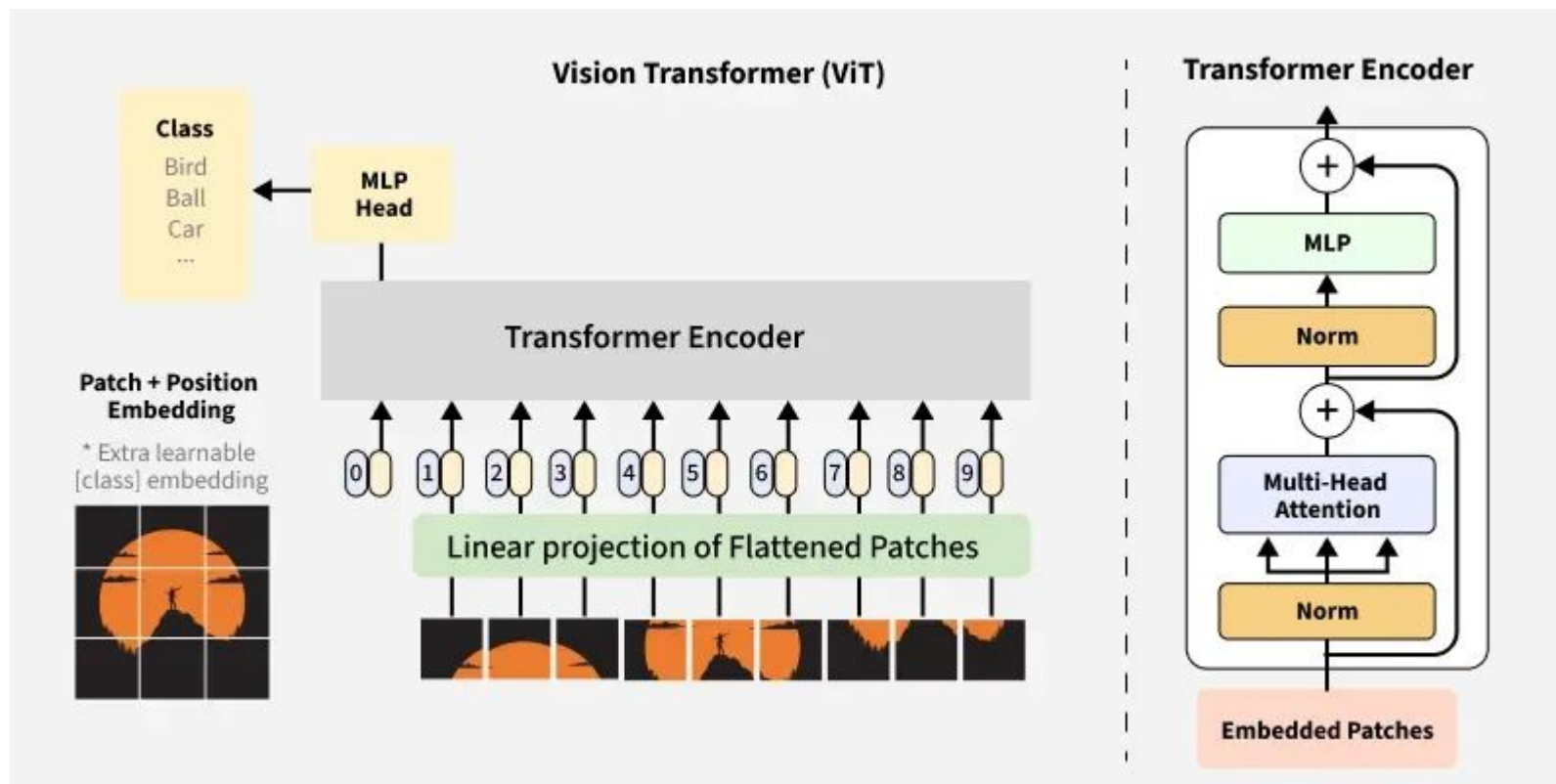
Classification – Object Detection – Recognition –
Machine training techniques
(Machine learning and Deep Learning)

Vision Transformer in Computer Vision

Vision Transformers (ViTs) are inspired by the success of transformers in NLP and apply self-attention mechanisms to interpret images by treating them as sequences of words. ViTs have found applications in various fields such as image classification, object detection, and segmentation.

How Vision Transformers Work?

ViTs work by dividing images into smaller patches, each of which is processed through self-attention layers to capture relationships between them.



Vision Transformer in Computer Vision

The process follows these key steps:

- 1.Patch Processing:** The image is divided into patches, and each patch is transformed into a vector.
- 2.Positional Encoding:** Unlike CNNs, ViTs do not have inherent spatial hierarchies. Positional information is added to the patches to preserve their relative positions.
- 3.Self-Attention Mechanism:** The model analyzes the interactions between patches, allowing it to focus on relevant parts of the image.
- 4.Final Prediction:** The output from the self-attention layers is used to classify or predict the desired result.

Vision Transformers vs. Convolutional Neural Networks

Vision Transformers and Convolutional Neural Networks (CNNs) tackle image processing with fundamentally distinct methodologies.

	Vision Transformers	Convolutional Neural Networks
Feature Extraction	Global feature extraction uses self-attention	Local feature extraction uses convolutional filters
Inductive Bias	Minimal inductive bias relies on data	Strong inductive bias includes built-in translation invariance
Data Requirement	High data requirements, large-scale datasets are necessary	Lower requirements can still be effective for smaller datasets
Computational Cost	High because of self-attention	Reduced by using convolutions for optimization
Scalability	Improvement in scalability as more data is added	Saturates after a specific dataset size

Building Vision Transformer from Scratch

Step 1: Import Libraries

We import the necessary libraries for data handling, model building, and visualization.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import transforms
from torchvision.datasets import OxfordIIITPet
import matplotlib.pyplot as plt
import numpy as np
```

Step 2: Define Hyperparameters

We set the parameters required for the model, such as the number of classes, image size, patch size, and learning rate.

```
# Define hyperparameters
num_classes = 37 # Oxford-IIIT Pet Dataset has 37 categories
image_size = 128
patch_size = 16
num_patches = (image_size // patch_size) ** 2
projection_dim = 64
num_heads = 4
transformer_units = [projection_dim * 2, projection_dim]
transformer_layers = 8
mlp_head_units = [2048, 1024]
learning_rate = 0.001
weight_decay = 0.0001
batch_size = 32
num_epochs = 50
```

Building Vision Transformer from Scratch

Step 3: Prepare the Data

We use `torchvision.transforms` to preprocess the data and load the Oxford-IIIT Pet dataset for training and testing.

```
# Data preparation
transform = transforms.Compose([
    transforms.Resize((image_size, image_size)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)),
])

trainset = OxfordIIITPet(root='./data', split='trainval',
    target_types='category', download=True, transform=transform)
trainloader = DataLoader(trainset, batch_size=batch_size, shuffle=True)

testset = OxfordIIITPet(root='./data', split='test', target_types='category',
    download=True, transform=transform)
testloader = DataLoader(testset, batch_size=batch_size, shuffle=False)
```

Building Vision Transformer from Scratch

Step 4: Define MLP

The Multi-Layer Perceptron (MLP) will be used in both the transformer blocks and the final classification head.

```
# Define MLP
class MLP(nn.Module):
    def __init__(self, hidden_units, dropout_rate):
        super(MLP, self).__init__()
        layers = []
        for units in hidden_units:
            layers.append(nn.Linear(units[0], units[1]))
            layers.append(nn.GELU())
            layers.append(nn.Dropout(dropout_rate))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)
```

Building Vision Transformer from Scratch

Step 5: Define Patch Embedding

This layer divides the input image into patches and projects them into a higher-dimensional space for processing.

```
# Define Patch Embedding
class PatchEmbedding(nn.Module):
    def __init__(self, image_size, patch_size, embed_dim):
        super(PatchEmbedding, self).__init__()
        self.patch_size = patch_size
        self.num_patches = (image_size // patch_size) ** 2
        self.projection = nn.Conv2d(3, embed_dim, kernel_size=patch_size,
stride=patch_size)

    def forward(self, x):
        return self.projection(x).flatten(2).transpose(1, 2)
```

Building Vision Transformer from Scratch

Step 6: Define Transformer Block

Each transformer block applies self-attention and an MLP with residual connections and normalization.

```
# Define Transformer Block
class TransformerBlock(nn.Module):
    def __init__(self, embed_dim, num_heads, mlp_hidden_dim, dropout):
        super(TransformerBlock, self).__init__()
        self.attention = nn.MultiheadAttention(embed_dim, num_heads, dropout=dropout)
        self.norm1 = nn.LayerNorm(embed_dim)
        self.mlp = MLP([[embed_dim, mlp_hidden_dim], [mlp_hidden_dim, embed_dim]], dropout)
        self.norm2 = nn.LayerNorm(embed_dim)

    def forward(self, x):
        attention_output = self.attention(x, x, x)[0]
        x = self.norm1(x + attention_output)
        mlp_output = self.mlp(x)
        x = self.norm2(x + mlp_output)
        return x
```

Building Vision Transformer from Scratch

Step 7: Define Vision Transformer

The Vision Transformer model consists of patch embedding, transformer blocks, and a classification head.

```
# Define Vision Transformer (ViT)
class VisionTransformer(nn.Module):
    def __init__(self, num_classes, image_size, patch_size, embed_dim, num_heads, transformer_units,
num_layers, mlp_head_units):
        super(VisionTransformer, self).__init__()
        self.patch_embedding = PatchEmbedding(image_size, patch_size, embed_dim)
        self.position_embedding = nn.Parameter(torch.zeros(1, self.patch_embedding.num_patches, embed_dim))
        self.transformer_blocks = nn.ModuleList([
            TransformerBlock(embed_dim, num_heads, transformer_units[0], 0.1)
            for _ in range(num_layers)
        ])
        self.mlp_head = MLP([[embed_dim, mlp_head_units[0]], [mlp_head_units[0], mlp_head_units[1]]], 0.5)
        self.classifier = nn.Linear(mlp_head_units[1], num_classes)

    def forward(self, x):
        x = self.patch_embedding(x)
        x = x + self.position_embedding
        for block in self.transformer_blocks:
            x = block(x)
        x = x.mean(dim=1)
        x = self.mlp_head(x)
        x = self.classifier(x)
        return x
```

Building Vision Transformer from Scratch

Step 8: Train the Model

We train the model using AdamW optimizer and cross-entropy loss.

```
# Instantiate the model
model = VisionTransformer(
    num_classes=num_classes,
    image_size=image_size,
    patch_size=patch_size,
    embed_dim=projection_dim,
    num_heads=num_heads,
    transformer_units=transformer_units,
    num_layers=transformer_layers,
    mlp_head_units=mlp_head_units
)

# Training loop
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=learning_rate, weight_decay=weight_decay)

for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    for batch_idx, (images, labels) in enumerate(trainloader):
        images, labels = images.to('cuda'), labels.to('cuda')
        model = model.to('cuda')

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    print(f"Epoch {epoch+1}/{num_epochs}, Loss: {running_loss/len(trainloader):.4f}")
```

Building Vision Transformer from Scratch

Step 9: Evaluate the Model

We evaluate the model on the test set and visualize predictions.

```
# Evaluation loop with visualization
model.eval()
correct = 0
total = 0

with torch.no_grad():
    images, labels = next(iter(testloader))
    images, labels = images.to('cuda'), labels.to('cuda')
    outputs = model(images)
    _, predicted = torch.max(outputs, 1)

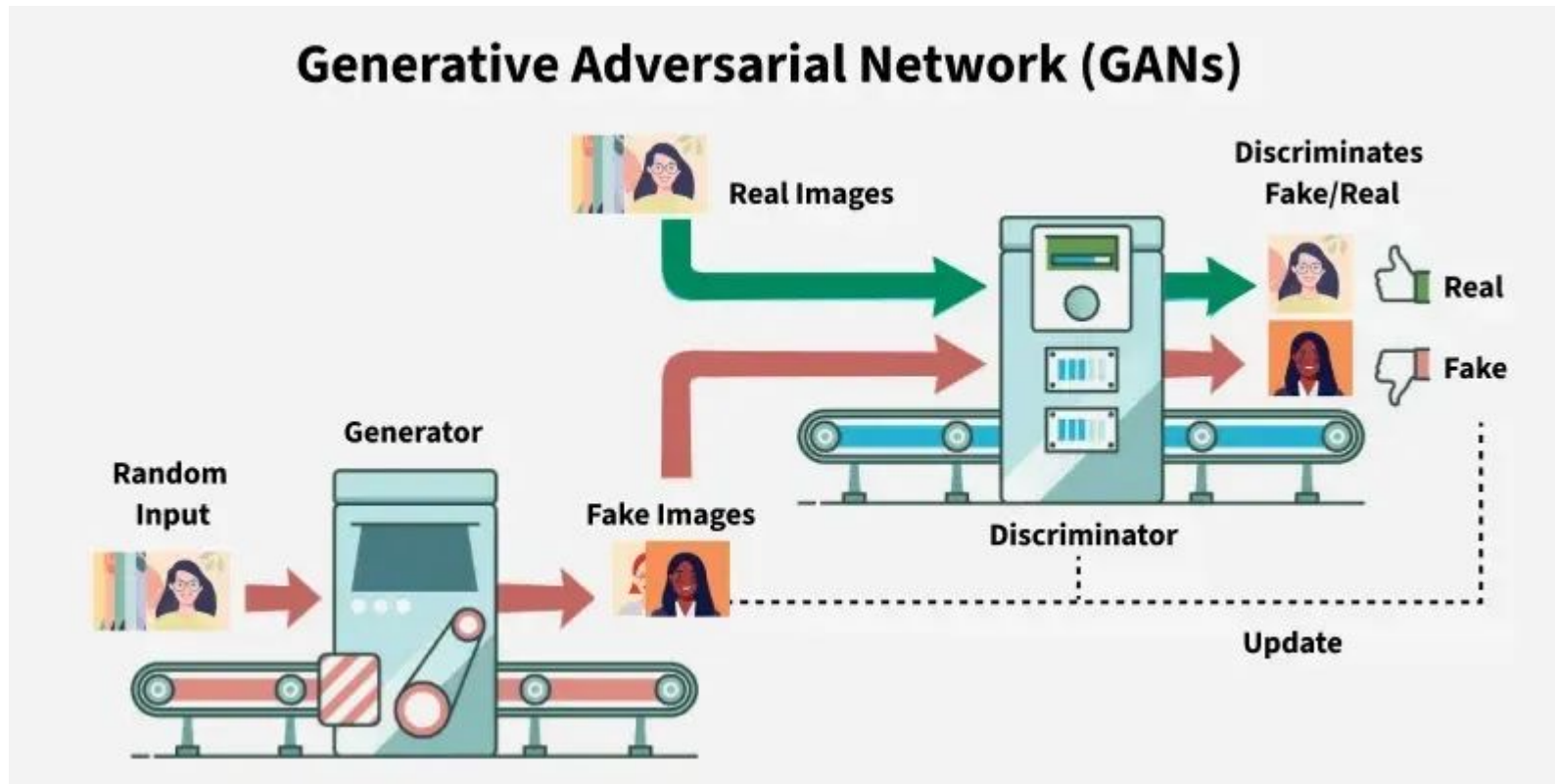
    # Visualize predictions
    fig, axes = plt.subplots(1, 5, figsize=(15, 5))
    for i in range(5):
        img = images[i].permute(1, 2, 0).cpu().numpy() * 0.5 + 0.5 # Unnormalize
        axes[i].imshow(img)
        axes[i].set_title(f"Predicted: {trainset.classes[predicted[i]]}\nActual: {trainset.classes[labels[i]]}")
        axes[i].axis("off")
    plt.show()

    total += labels.size(0)
    correct += (predicted == labels).sum().item()

print(f"Accuracy: {100 * correct / total:.2f}%")
```

Generative Adversarial Network (GAN)

Generative Adversarial Networks (GAN) help machines to create new, realistic data by learning from existing examples. It is introduced by Ian Goodfellow and his team in 2014 and they have transformed how computers generate images, videos, music and more. Unlike traditional models that only recognize or classify data, they take a creative way by generating entirely new content that closely resembles real-world data. This ability helped various fields such as art, gaming, healthcare and data science. In this article, we will see more about GANs and its core concepts.



Generative Adversarial Network (GAN)

Architecture of GAN

GAN consist of two main models that work together to create realistic synthetic data which are as follows:

1. Generator Model

The generator is a deep neural network that takes random noise as input to generate realistic data samples like images or text. It learns the underlying data patterns by adjusting its internal parameters during training through [backpropagation](#). Its objective is to produce samples that the discriminator classifies as real.

Generator Loss Function: The generator tries to minimize this loss:

$$J_G = -\frac{1}{m} \sum_{i=1}^m \log D(G(z_i))$$

where

- J_G measure how well the generator is fooling the discriminator.
- $G(z_i)$ is the generated sample from random noise z_i
- $D(G(z_i))$ is the discriminator's estimated probability that the generated sample is real.

The generator aims to maximize $D(G(z_i))$ meaning it wants the discriminator to classify its fake data as real (probability close to 1).

Generative Adversarial Network (GAN)

2. Discriminator Model

The discriminator acts as a binary classifier helps in distinguishing between real and generated data. It learns to improve its classification ability through training, refining its parameters to detect fake samples more accurately. When dealing with image data, the discriminator uses convolutional layers or other relevant architectures which help to extract features and enhance the model's ability.

Discriminator Loss Function: The discriminator tries to minimize this loss:

$$J_D = -\frac{1}{m} \sum_{i=1}^m \log D(x_i) - \frac{1}{m} \sum_{i=1}^m \log(1 - D(G(z_i)))$$

- J_D measures how well the discriminator classifies real and fake samples.
- x_i is a real data sample.
- $G(z_i)$ is a fake sample from the generator.
- $D(x_i)$ is the discriminator's probability that x_i is real.
- $D(G(z_i))$ is the discriminator's probability that the fake sample is real.

The discriminator wants to correctly classify real data as real (maximize $\log D(x_i)$) and fake data as fake (maximize $\log(1 - D(G(z_i)))$)

Generative Adversarial Network (GAN)

MinMax Loss

GANs are trained using a [MinMax Loss](#) between the generator and discriminator:

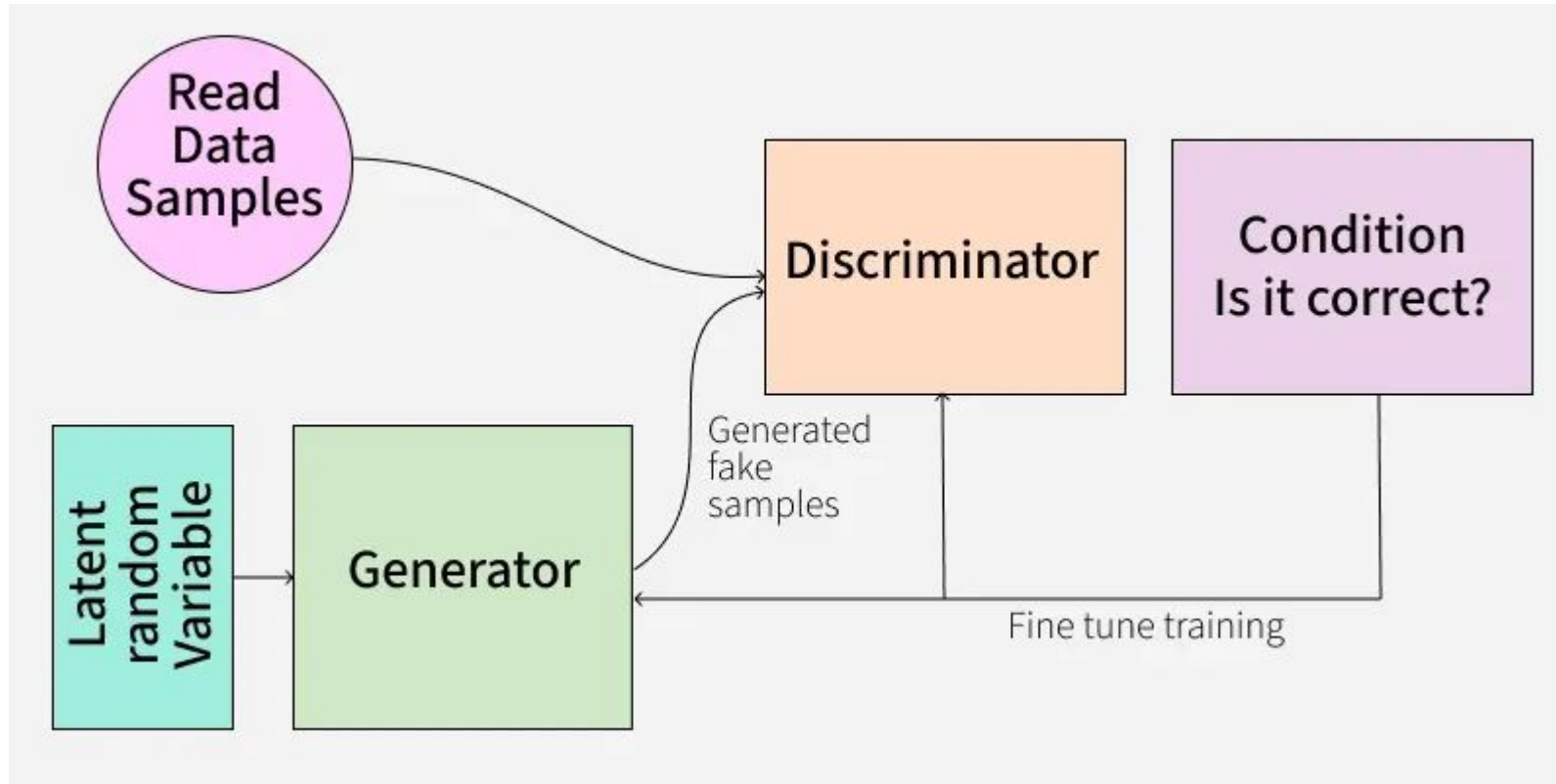
$$\min_G \max_D (G, D) = [\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(g(z)))]$$

where,

- G is generator network and D is the discriminator network
- $p_{data}(x)$ = true data distribution
- $p_z(z)$ = distribution of random noise (usually normal or uniform)
- $D(x)$ = discriminator's estimate of real data
- $D(G(z))$ = discriminator's estimate of generated data

The generator tries to minimize this loss (to fool the discriminator) and the discriminator tries to maximize it (to detect fakes accurately).

Generative Adversarial Network (GAN)



Generative Adversarial Network (GAN)

How does a GAN work?

GAN train by having two networks the Generator (G) and the Discriminator (D) compete and improve together. Here's the step-by-step process

1. Generator's First Move

The generator starts with a random noise vector like random numbers. It uses this noise as a starting point to create a fake data sample such as a generated image. The generator's internal layers transform this noise into something that looks like real data.

2. Discriminator's Turn

The discriminator receives two types of data:

- Real samples from the actual training dataset.
- Fake samples created by the generator.

D's job is to analyze each input and find whether it's real data or something G cooked up. It outputs a probability score between 0 and 1. A score of 1 shows the data is likely real and 0 suggests it's fake.

3. Adversarial Learning

- If the discriminator correctly classifies real and fake data it gets better at its job.
- If the generator fools the discriminator by creating realistic fake data, it receives a positive update and the discriminator is penalized for making a wrong decision.

4. Generator's Improvement

- Each time the discriminator mistakes fake data for real, the generator learns from this success.
- Through many iterations, the generator improves and creates more convincing fake samples.

5. Discriminator's Adaptation

- The discriminator also learns continuously by updating itself to better spot fake data.
- This constant back-and-forth makes both networks stronger over time.

6. Training Progression

- As training continues, the generator becomes highly proficient at producing realistic data.
- Eventually the discriminator struggles to distinguish real from fake shows that the GAN has reached a well-trained state.
- At this point, the generator can produce high-quality synthetic data that can be used for different applications.

Generative Adversarial Network (GAN)

Types of GAN

There are several types of GANs each designed for different purposes. Here are some important types:

1. Vanilla GAN

Vanilla GAN is the simplest type of GAN. It consists of:

- A generator and a discriminator both are built using multi-layer perceptrons (MLPs).
- The model optimizes its mathematical formulation using stochastic gradient descent (SGD).
- While foundational, Vanilla GAN can face problems like:

Mode collapse: The generator produces limited types of outputs repeatedly.

Unstable training: The generator and discriminator may not improve smoothly.

2. Conditional GAN (CGAN)

Conditional GAN (CGAN) adds an additional conditional parameter to guide the generation process. Instead of generating data randomly they allow the model to produce specific types of outputs.

Working of CGANs:

- A conditional variable (y) is fed into both the generator and the discriminator.
- This ensures that the generator creates data corresponding to the given condition (e.g generating images of specific objects).
- The discriminator also receives the labels to help distinguish between real and fake data.
- Example: Instead of generating any random image, CGAN can generate a specific object like a dog or a cat based on the label.

Generative Adversarial Network (GAN)

3. Deep Convolutional GAN (DCGAN)

Deep Convolutional GAN (DCGAN) are among the most popular types of GANs used for image generation.

They are important because they:

- Uses Convolutional Neural Networks (CNNs) instead of simple multi-layer perceptrons (MLPs).
- Max pooling layers are replaced with convolutional stride helps in making the model more efficient.
- Fully connected layers are removed, which allows for better spatial understanding of images.
- DCGANs are successful because they generate high-quality, realistic images.

4. Laplacian Pyramid GAN (LAPGAN)

Laplacian Pyramid GAN (LAPGAN) is designed to generate ultra-high-quality images by using a multi-resolution approach.

Working of LAPGAN:

- Uses multiple generator-discriminator pairs at different levels of the Laplacian pyramid.
- Images are first down sampled at each layer of the pyramid and upsampled again using Conditional GAN (CGAN).
- This process allows the image to gradually refine details and helps in reducing noise and improving clarity.
- Due to its ability to generate highly detailed images, LAPGAN is considered a superior approach for photorealistic image generation.

Generative Adversarial Network (GAN)

5. Super Resolution GAN (SRGAN)

Super-Resolution GAN (SRGAN) is designed to increase the resolution of low-quality images while preserving details.

Working of SRGAN:

- Uses a deep neural network combined with an adversarial loss function.
- Enhances low-resolution images by adding finer details helps in making them appear sharper and more realistic.
- Helps to reduce common image upscaling errors such as blurriness and pixelation.

Hidden Markov Model

- When working with sequences of data, we often face situations **where we can't directly see the important factors that influence the datasets. Hidden Markov Models (HMM)** help solve this problem by predicting these hidden factors based on the observable data.

Hidden Markov Model in Machine Learning

- It is an [statistical model](#) that is used to describe the **probabilistic relationship between a sequence of observations and a sequence of hidden states**. like it is often used in situations where the underlying system or process that generates the observations is unknown or hidden, hence it has the name "**Hidden Markov Model**."

Hidden Markov Model

- An HMM consists of two types of variables: hidden states and observations.
- The **hidden states** are the underlying variables that generate the observed data, but they are not directly observable.
- The **observations** are the variables that are measured and observed.
- The relationship between the hidden states and the observations is modelled using a probability distribution. The Hidden Markov Model (HMM) describes the relationship between the hidden states and the observations, utilising two sets of probabilities: the transition probabilities and the emission probabilities.
- The **transition probabilities** describe the probability of transitioning from one hidden state to another.
- The **emission probabilities** describe the probability of observing an output given a hidden state.

Hidden Markov Model

Hidden Markov Model Algorithm

- The Hidden Markov Model (HMM) algorithm can be implemented using the following steps:

Step 1: Define the state space and observation space: The state space is the set of all possible hidden states, and the observation space is the set of all possible observations.

Step 2: Define the initial state distribution: This is the probability distribution over the initial state.

Step 3: Define the state transition probabilities: These are the probabilities of transitioning from one state to another. This forms the transition matrix, which describes the probability of moving from one state to another.

Step 4: Define the observation likelihoods: These are the probabilities of generating each observation from each state. This forms the emission matrix, which describes the probability of generating each observation from each state.

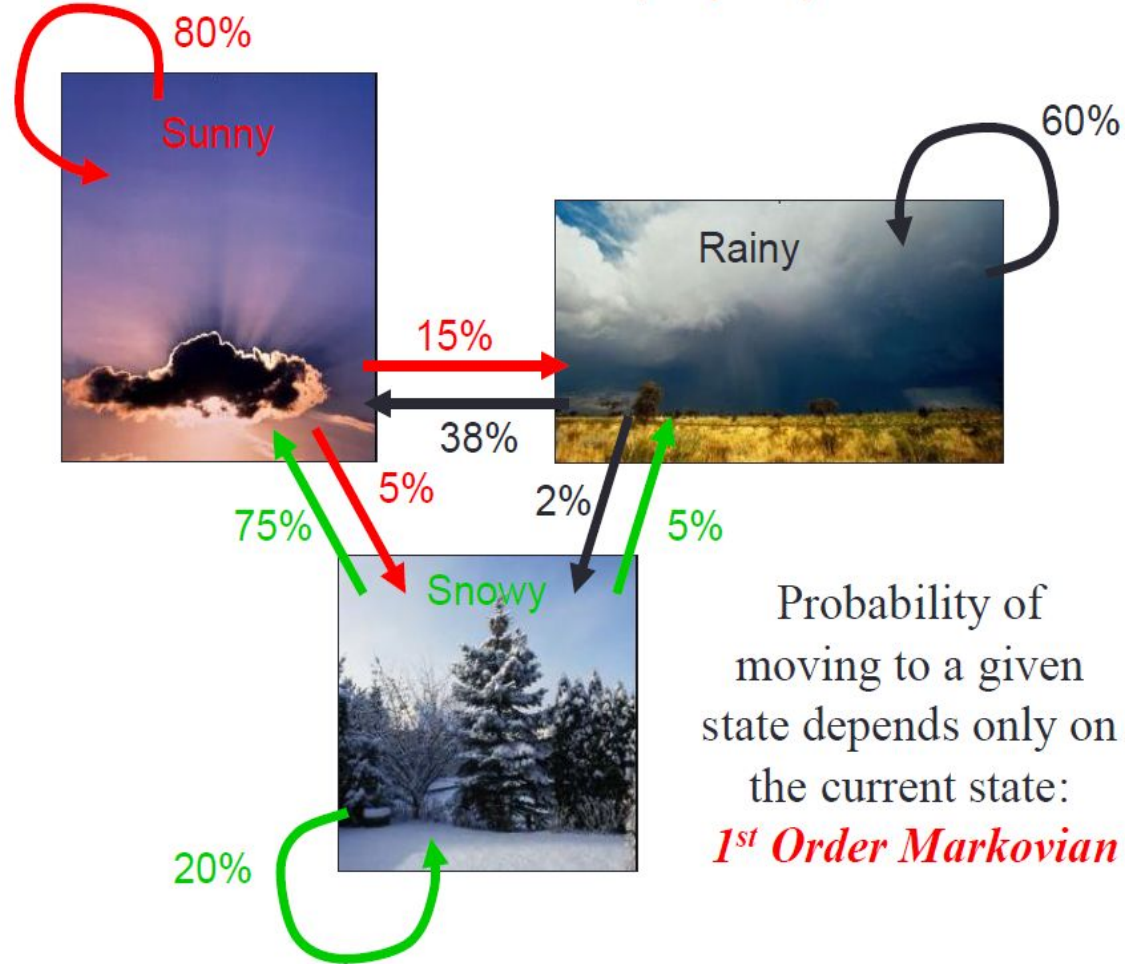
Step 5: Train the model: The parameters of the state transition probabilities and the observation likelihoods are estimated using the Baum-Welch algorithm, or the forward-backward algorithm. This is done by iteratively updating the parameters until convergence.

Step 6: Decode the most likely sequence of hidden states: Given the observed data, the Viterbi algorithm is used to compute the most likely sequence of hidden states. This can be used to predict future observations, classify sequences, or detect patterns in sequential data.

Step 7: Evaluate the model: The performance of the HMM can be evaluated using various metrics, such as accuracy, precision, recall, or F1 score.

Hidden Markov Model

Weather: A Markov Model (*maybe?*)



Hidden Markov Model

Ingredients of a Markov Model

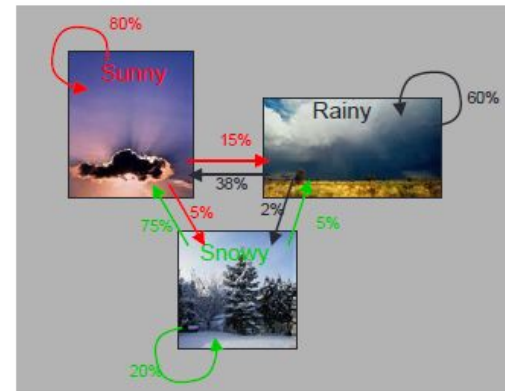
- States: $\{S_1, S_2, \dots, S_N\}$

- State transition probabilities:

$$a_{ij} = P(q_{t+1} = S_i \mid q_t = S_j)$$

- Initial state distribution:

$$\pi_i = P[q_1 = S_i]$$



Hidden Markov Model

Ingredients of Our Markov Model

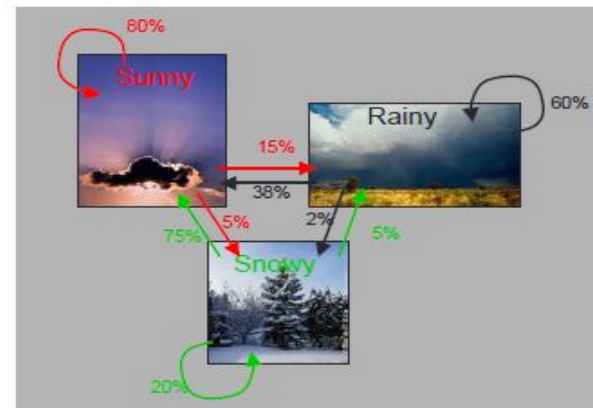
- States: $\{S_{\text{sunny}}, S_{\text{rainy}}, S_{\text{snowy}}\}$

- State transition probabilities:

$$A = \begin{pmatrix} .8 & .15 & .05 \\ .38 & .6 & .02 \\ .75 & .05 & .2 \end{pmatrix}$$

- Initial state distribution:

$$\pi = (.7 \quad .25 \quad .05)$$



Hidden Markov Model

Specification of an HMM

- A - the state transition probability matrix

- $a_{ij} = P(q_{t+1} = j | q_t = i)$

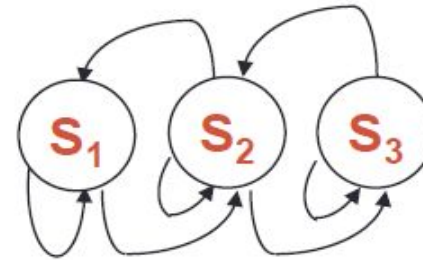
- B - observation probability distribution

- Discrete: $b_j(k) = P(o_t = k | q_t = j) \quad i \leq k \leq M$

- Continuous $b_j(x) = p(o_t = x | q_t = j)$

- π - the initial state distribution

- $\pi(j) = P(q_1 = j)$



- Full HMM over a of states and output space is thus specified as a triplet: $\lambda = (A, B, \pi)$

Gaussian Mixture Model

- **Gaussian Mixture Model** is a probabilistic model that assumes all data points are generated from a mixture of several Gaussian distributions with unknown parameters. Unlike hard clustering methods like K-Means that assigns each data point to one cluster based on the closest centroid which often doesn't align with the complexity of real-world data. **GMMs perform soft clustering meaning each data point belongs to multiple clusters with certain probabilities.** Each Gaussian in the mixture is defined by:
 - **Mean (μ):** The center of the distribution.
 - **Covariance (Σ):** Describes the spread and orientation.
 - **Mixing coefficient (π):** Represents the proportion of each Gaussian in the mixture.

Gaussian Mixture Model

How Gaussian Mixture Models Work?

- Now that we understand the key differences between K-Means and GMM let's go deeper into how GMM actually works. Here Each cluster is represented by a Gaussian distribution and the data points are assigned probabilities of belonging to different clusters based on their distance from each Gaussian.
- The Gaussian mixture model assigns a probability to each data point x_n of belonging to a cluster. The probability of data point coming from Gaussian cluster k is expressed as

$$P(z_n = k \mid x_n) = \frac{\pi_k \cdot \mathcal{N}(x_n \mid \mu_k, \Sigma_k)}{\sum_{k=1}^K \pi_k \cdot \mathcal{N}(x_n \mid \mu_k, \Sigma_k)}$$

where:

- $z_n=k$ is a latent variable indicating which Gaussian the point belongs to.
- π_k is the mixing probability of the k -th Gaussian.
- $\mathcal{N}(x_n \mid \mu_k, \Sigma_k)$ is the Gaussian distribution with mean μ_k and covariance Σ_k

Next we need to calculate the overall likelihood of observing a data point x_n under all Gaussians. This is achieved by summing over all possible clusters (Gaussians) for each point:

$$P(x_n) = \sum_{k=1}^K \pi_k \mathcal{N}(x_n \mid \mu_k, \Sigma_k)$$

where:

- $P(x_n)$ is the overall likelihood of observing the data point x_n
- The sum accounts for all possible Gaussians k .

Gaussian Mixture Model

The Expectation-Maximization (EM) Algorithm

To fit a Gaussian Mixture Model to the data we use the Expectation-Maximization (EM) algorithm which is an iterative method that optimizes the parameters of the Gaussian distributions like mean, covariance and mixing coefficients. It works in two main steps:

- **Expectation Step (E-step):** In this step the algorithm calculates the probability that each data point belongs to each cluster based on the current parameter estimates (mean, covariance, mixing coefficients).
- **Maximization Step (M-step):** After estimating the probabilities the algorithm updates the parameters (mean, covariance and mixing coefficients) to better fit the data.

These two steps are repeated until the model converges meaning the parameters no longer change significantly between iterations. Here's a simple breakdown of the GMM process:

1. **Initialization:** Start with initial guesses for the means, covariances and mixing coefficients of each Gaussian distribution.
2. **E-step:** For each data point, calculate the probability of it belonging to each Gaussian distribution (cluster).
3. **M-step:** Update the parameters (means, covariances, mixing coefficients) using the probabilities calculated in the E-step.
4. **Repeat:** Continue alternating between the E-step and M-step until the log-likelihood of the data (a measure of how well the model fits the data) converges.

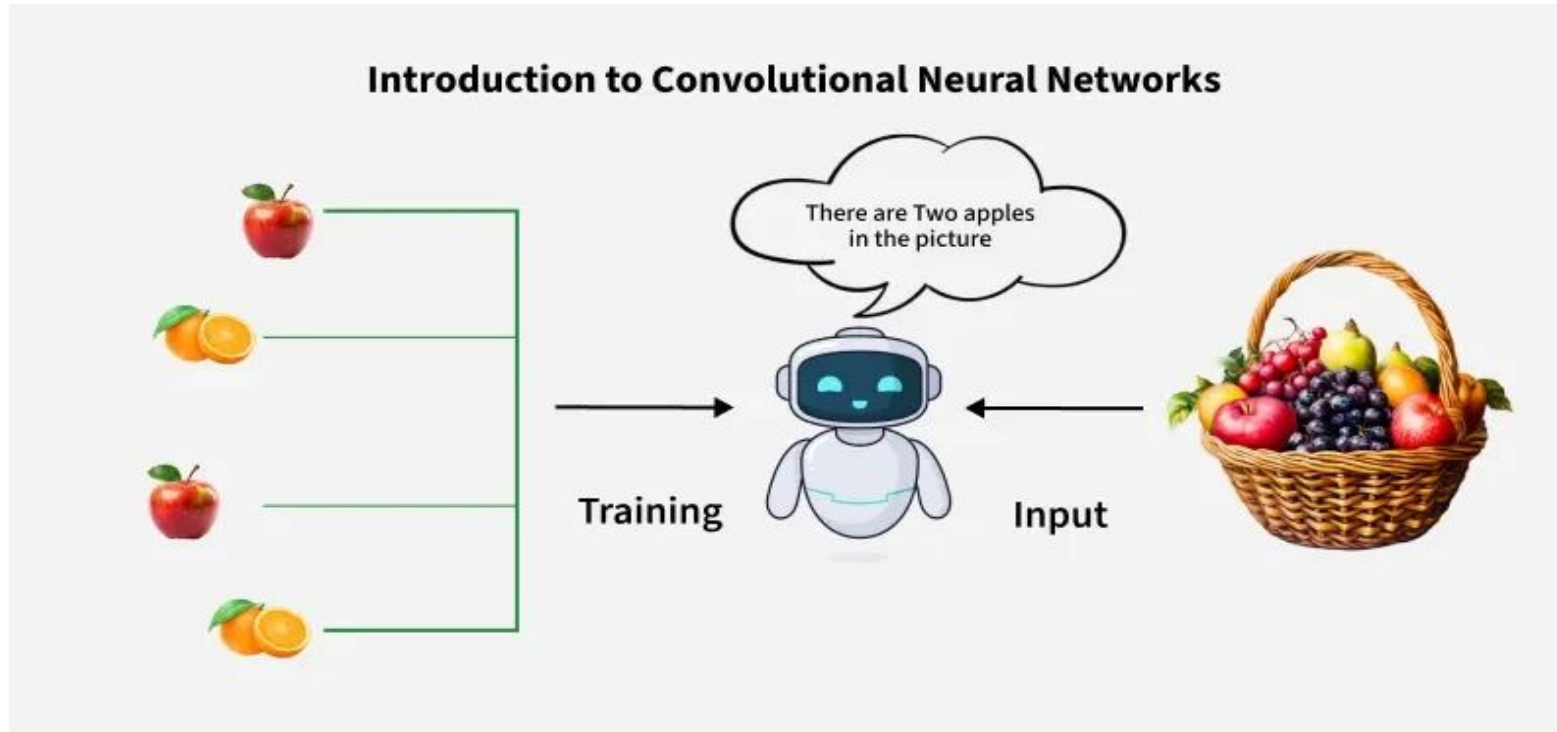
Formula:

$$L(\mu_k, \Sigma_k, \pi_k) = \prod_{n=1}^N \sum_{k=1}^K \pi_k \mathcal{N}(x_n | \mu_k, \Sigma_k)$$

The E-step computes the probabilities that each data point belongs to each Gaussian while the M-step updates the parameters μ_k , Σ_k and π_k based on these probabilities.

Convolutional Neural Network (CNN)

- [Convolutional Neural Networks \(CNNs\)](#) are deep learning models designed to process data with a grid-like topology such as images. They are the foundation for most modern computer vision applications to detect features within visual data.



Convolutional Neural Network (CNN)

Key Components of a Convolutional Neural Network

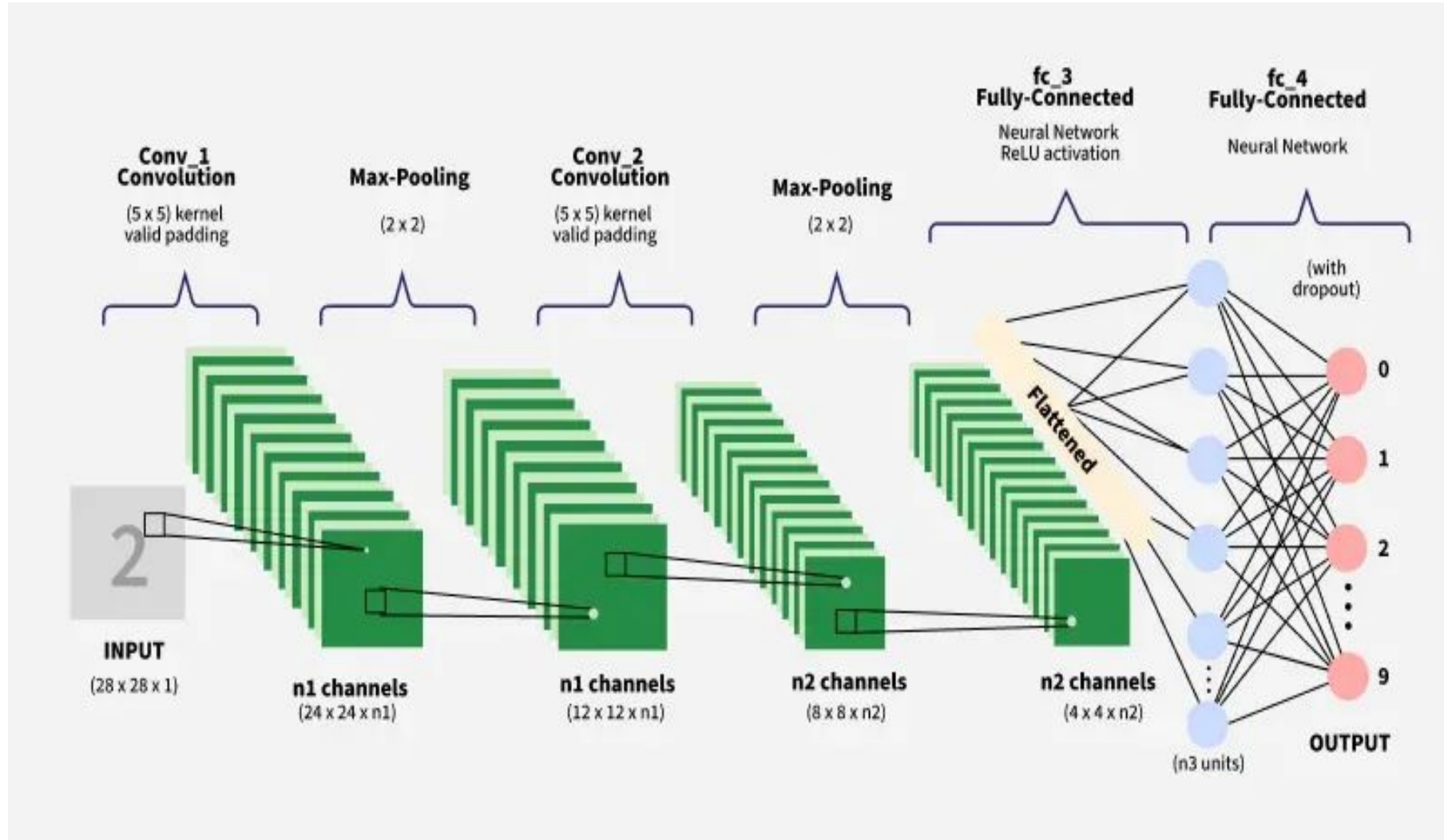
1. **Convolutional Layers**: These layers apply convolutional operations to input images using filters or kernels to detect features such as edges, textures and more complex patterns. Convolutional operations help preserve the spatial relationships between pixels.
2. **Pooling Layers**: They downsample the spatial dimensions of the input, reducing the computational complexity and the number of parameters in the network. Max pooling is a common pooling operation where we select a maximum value from a group of neighboring pixels.
3. **Activation Functions**: They introduce non-linearity to the model by allowing it to learn more complex relationships in the data.
4. **Fully Connected Layers**: These layers are responsible for making predictions based on the high-level features learned by the previous layers. They connect every neuron in one layer to every neuron in the next layer.

Convolutional Neural Network (CNN)

How CNNs Work?

- 1.Input Image:** CNN receives an input image which is preprocessed to ensure uniformity in size and format.
- 2.Convolutional Layers:** Filters are applied to the input image to extract features like edges, textures and shapes.
- 3.Pooling Layers:** The feature maps generated by the convolutional layers are downsampled to reduce dimensionality.
- 4.Fully Connected Layers:** The downsampled feature maps are passed through fully connected layers to produce the final output, such as a classification label.
- 5.Output:** The CNN outputs a prediction, such as the class of the image.

Convolutional Neural Network (CNN)



Convolutional Neural Network (CNN)

How to Train a Convolutional Neural Network?

CNNs are trained using a supervised learning approach. This means that the CNN is given a set of labeled training images. The CNN learns to map the input images to their correct labels.

The training process for a CNN involves the following steps:

- 1.Data Preparation:** The training images are preprocessed to ensure that they are all in the same format and size.
- 2.Loss Function:** A loss function is used to measure how well the CNN is performing on the training data. The loss function is typically calculated by taking the difference between the predicted labels and the actual labels of the training images.
- 3.Optimizer:** An optimizer is used to update the weights of the CNN in order to minimize the loss function.
- 4.Backpropagation:** Backpropagation is a technique used to calculate the gradients of the loss function with respect to the weights of the CNN. The gradients are then used to update the weights of the CNN using the optimizer.

Convolutional Neural Network (CNN)

How to Evaluate CNN Models

Efficiency of CNN can be evaluated using a variety of criteria. Among the most popular metrics are:

- Accuracy:** Accuracy is the percentage of test images that the CNN correctly classifies.
- Precision:** Precision is the percentage of test images that the CNN predicts as a particular class and that are actually of that class.
- Recall:** Recall is the percentage of test images that are of a particular class and that the CNN predicts as that class.
- F1 Score:** The F1 Score is a harmonic mean of precision and recall. It is a good metric for evaluating the performance of a CNN on classes that are imbalanced.

Convolutional Neural Network (CNN)

How to Evaluate CNN Models

Efficiency of CNN can be evaluated using a variety of criteria. Among the most popular metrics are:

- **Accuracy:** Accuracy is the percentage of test images that the CNN correctly classifies.
- **Precision:** Precision is the percentage of test images that the CNN predicts as a particular class and that are actually of that class.
- **Recall:** Recall is the percentage of test images that are of a particular class and that the CNN predicts as that class.
- **F1 Score:** The F1 Score is a harmonic mean of precision and recall. It is a good metric for evaluating the performance of a CNN on classes that are imbalanced.

		POSITIVE	NEGATIVE
ACTUAL VALUES	POSITIVE	TP	FN
	NEGATIVE	FP	TN

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$Accuracy = \frac{TP + TN}{TP + FP + FN + TN}$$

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Convolutional Neural Network (CNN)

Different Types of CNN Models

1. LeNet

[LeNet](#) developed by Yann LeCun and his colleagues in the late 1990s was one of the first successful CNNs designed for handwritten digit recognition. It laid the foundation for modern CNNs and achieved high accuracy on the MNIST dataset which contains 70,000 images of handwritten digits (0-9).

2. AlexNet

[AlexNet](#) is a CNN architecture that was developed by Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton in 2012. It was the first CNN to win the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) a major image recognition competition. It consists of several layers of convolutional and pooling layers followed by fully connected layers. The architecture includes five convolutional layers, three pooling layers and three fully connected layers.

3. Resnet

[ResNets \(Residual Networks\)](#) are designed for image recognition and processing tasks. They are renowned for their ability to train very deep networks without overfitting making them highly effective for complex tasks. It introduces skip connections that allow the network to learn residual functions making it easier to train deep architecture.

Convolutional Neural Network (CNN)

4. GoogleNet

GoogleNet also known as InceptionNet is renowned for achieving high accuracy in image classification while using fewer parameters and computational resources compared to other state-of-the-art CNNs. The core component of GoogleNet allows the network to learn features at different scales simultaneously to enhance performance.

5. VGG

VGGs are developed by the Visual Geometry Group at Oxford, it uses small 3x3 convolutional filters stacked in multiple layers, creating a deep and uniform structure. Popular variants like VGG-16 and VGG-19 achieved state-of-the-art performance on the ImageNet dataset demonstrating the power of depth in CNNs.

Convolutional Neural Network (CNN)

Applications of CNN

- **Image classification:** CNNs are the state-of-the-art models for image classification. They can be used to classify images into different categories such as cats and dogs.
- **Object detection:** It can be used to detect objects in images such as people, cars and buildings. They can also be used to localize objects in images which means that they can identify the location of an object in an image.
- **Image segmentation:** It can be used to segment images which means that they can identify and label different objects in an image. This is useful for applications such as medical imaging and robotics.
- **Video analysis:** It can be used to analyze videos such as tracking objects in a video or detecting events in a video. This is useful for applications such as video surveillance and traffic monitoring.

Advantages of CNN

- **High Accuracy:** They can achieve high accuracy in various image recognition tasks.
- **Efficiency:** They are efficient, especially when implemented on GPUs.
- **Robustness:** They are robust to noise and variations in input data.
- **Adaptability:** It can be adapted to different tasks by modifying their architecture.

Disadvantages of CNN

- **Complexity:** It can be complex and difficult to train, especially for large datasets.
- **Resource-Intensive:** It requires significant computational resources for training and deployment.
- **Data Requirements:** They need large amounts of labeled data for training.
- **Interpretability:** They can be difficult to interpret making it challenging to understand their predictions.